

基于光学效率分解与同心圆布局搜索的定日镜场优化设计

摘要

针对塔式太阳能热发电站定日镜场的光学效率评价与布局优化问题，本文建立了“太阳位置一定日镜法向—光学效率分量—年平均输出热功率”的完整计算链条。光学效率分解为阴影遮挡效率、余弦效率、大气透射率、集热器截断效率与镜面反射率之积；年平均指标按每月 21 日五个代表时刻等权平均得到。对问题一，在吸收塔位于圆心、定日镜尺寸 $6\text{ m} \times 6\text{ m}$ 、安装高度 4 m 且位置由附件给定的条件下，求得年平均光学效率为 0.5668，年平均输出热功率为 34.60 MW ，单位镜面面积年平均输出热功率为 0.5508 kW/m^2 。对问题二，在额定功率不低于 60 MW 、定日镜尺寸与安装高度统一的约束下，以同心圆排布为骨架对塔位、镜面边长与安装高度进行网格搜索，得到边长 7.5 m 、安装高度 4.15 m 、数目 2226 的设计方案，年平均功率 60.74 MW ，单位面积年平均功率 0.4851 kW/m^2 。对问题三，允许各环尺寸与高度不同，在同样额定功率约束下将单位面积年平均功率提升至 0.5112 kW/m^2 （相对问题二约提高 5.4%）。结果表明：在额定功率约束下，外圈低效率镜面的引入会拉低单位面积指标，而分环变尺寸可在接近额定功率时改善截断与遮挡损失。

关键字： 定日镜场 光学效率 余弦效率 截断效率 同心圆布局 单目标优化

Abstract

We model heliostat-field annual optical efficiency and thermal power via sun position, mirror normals and a product of shading, cosine, atmospheric, intercept and reflectivity factors, averaged over five daily samples on the 21st of each month. Problem 1 yields annual optical efficiency 0.5668, power 34.60 MW and unit-area power 0.5508 kW/m². Problem 2 designs a uniform-size concentric layout meeting 60 MW, obtaining 7.5 m mirrors at 4.15 m height ($N = 2226$), 60.74 MW and 0.4851 kW/m². Problem 3 allows ring-wise size/height and improves unit-area power to 0.5112 kW/m² ($\approx 5.4\%$ over Problem 2). Rated-power constraints force outer low-efficiency mirrors; ring-wise sizing mitigates intercept/shading losses near the rating. **Keywords:** heliostat field; optical efficiency; cosine efficiency; intercept efficiency; concentric layout; single-objective optimization

插图清单

1	太阳高度角与方位角的年变化	11
2	问题一：定日镜场初始布局（塔在圆心）	14
3	问题一：各月平均光学效率及其分量	14
4	问题一：春分正午光学效率空间分布	15
5	问题一：各定日镜年平均光学效率（由内层起编号）	15
6	问题二：优化后定日镜场布局	17
7	问题二：各月平均效率	17
8	问题二：春分正午光学效率分布	18
9	问题二：随机可行解与最优解的单位面积功率比较	18
10	问题三：变尺寸/高度优化布局	20
11	问题三：各月平均效率	21
12	问题三：春分正午光学效率分布	21
13	问题三：定日镜尺寸随径向距离的变化	22
14	三问年均效率分量与单位面积功率对比	23
15	三问年平均输出热功率对比（虚线为额定 60 MW）	23

附表清单

1	主要符号说明	9
2	问题一每月 21 日平均光学效率及输出功率	15
3	问题一年平均光学效率及功率	16
4	问题二设计参数	18
5	问题二年平均光学效率及功率	18
6	问题二每月 21 日平均光学效率及输出功率	19
7	问题三设计参数	20
8	问题三年平均光学效率及功率	21
9	问题三每月 21 日平均光学效率及输出功率	22
10	三问核心指标汇总	23

一、问题重述

1.1 问题背景

塔式太阳能光热发电是实现“碳达峰、碳中和”的重要技术路径之一。电站通过大量可双轴跟踪的定日镜，将太阳直射辐射反射并汇聚至吸收塔顶部集热器，加热传热工质，再经热力循环发电。定日镜场的年平均光学效率决定了可用太阳能中有多少能够真正到达集热器；单位镜面面积年平均输出热功率则直接关系到镜场投资的效费比。因此，在给定场地几何与设备参数下，如何科学评价既有布局、并在额定年平均输出热功率约束下优化塔位、镜面尺寸、安装高度与排布方式，具有明确的工程意义。

1.2 已知条件

赛题给定如下关键条件（坐标系以圆形镜场中心为原点，东为 x 正方向，北为 y 正方向，竖直向上为 z ）：

1. 场址：东经 98.5° ，北纬 39.4° ，海拔 3000 m ；
2. 场地：圆形定日镜场半径 $R = 350\text{ m}$ ；吸收塔周围 100 m 内需留作厂房与辅助设备用地；
3. 吸收塔与集热器：塔高 80 m ；顶部圆柱形集热器直径 7 m 、高 8 m ；
4. 定日镜：平面矩形，边长范围 $2 \sim 8\text{ m}$ （通常宽不小于高），安装高度 $2 \sim 6\text{ m}$ ，且绕水平轴旋转时不得触地；相邻底座中心间距至少比镜面宽度多 5 m ；
5. 镜面反射率可取常数 0.92 ；
6. 评价时刻：每月 21 日的 $9:00$ 、 $10:30$ 、 $12:00$ 、 $13:30$ 、 $15:00$ ，共 $12 \times 5 = 60$ 个代表时刻。

1.3 求解任务

1. **问题 1**：吸收塔建于圆心，定日镜尺寸均为 $6\text{ m} \times 6\text{ m}$ 、安装高度均为 4 m ，位置由附件给出（共 1745 面）。计算年平均光学效率、年平均输出热功率及单位镜面面积年平均输出热功率，并按表 1、表 2 格式给出月平均与年平均结果。
2. **问题 2**：额定年平均输出热功率为 60 MW ；所有定日镜尺寸与安装高度相同。设计吸收塔位置、定日镜尺寸、安装高度、数目与位置，使单位镜面面积年平均输出热功率尽量大；结果写入 `result2.xlsx`。
3. **问题 3**：允许定日镜尺寸与安装高度不同，额定功率仍为 60 MW ，重新设计各参数，使单位面积年平均输出热功率尽量大；结果写入 `result3.xlsx`。

二、问题分析

2.1 总体建模框架

本问题的本质是：在时变太阳位置驱动下，计算每面定日镜将直射辐射反射到集热器的有效能量份额，再对时间与镜面集合取平均，得到场级年平均指标；并在额定功率约束下对布局参数寻优。

据此，本文将求解过程分解为四个层次：

1. **天文层**：由日期、当地时间与纬度计算太阳高度角 α_s 、方位角 γ_s 及法向直接辐射辐照度 DNI ；
2. **几何光学层**：由太阳方向与“镜心—集热器中心”连线确定镜面法向，进而得到余弦效率；结合距离得到大气透射率；
3. **损失层**：计入塔影与邻镜遮挡（阴影遮挡效率），以及太阳光锥造成的光斑溢出（截断效率）；
4. **场级汇总与优化层**：对 60 个代表时刻求年平均功率 P_{year} 与单位面积指标 E ；问题二、三再在约束 $P_{\text{year}} \geq 60 \text{ MW}$ 下最大化 E 。

光学效率的标准分解为

$$\eta = \eta_{sb} \eta_{cos} \eta_{at} \eta_{trunc} \eta_{ref}, \quad (1)$$

其中 $\eta_{ref} = 0.92$ 为常数，其余四项均随时间与镜位变化。场瞬时输出热功率

$$P(t) = DNI(t) \sum_{i=1}^N A_i \eta_i(t), \quad A_i = w_i h_i, \quad (2)$$

年平均功率取 60 个代表时刻的算术平均。单位镜面面积年平均输出热功率

$$E = \frac{P_{\text{year}}}{\sum_{i=1}^N A_i}. \quad (3)$$

2.2 问题一的分析

问题一是**完全信息下的正向评价问题**：塔位、镜面尺寸、安装高度与全部镜位均已给定，不存在决策自由度。求解路径可严格写成算法流程：

1. 读入附件中 1745 个 (x_i, y_i) ，令 $w_i = h_i = 6 \text{ m}$ ， $z_i = 4 \text{ m}$ ，塔位 $(x_t, y_t) = (0, 0)$ ；
2. 对每月 21 日的五个时刻，计算 $(\alpha_s, \gamma_s, DNI)$ ；
3. 对每面镜计算 s 、 r 、 n ，得到 η_{cos} 、 η_{at} ，并估计 η_{sb} 、 η_{trunc} ；
4. 由式(1)–(2)得到瞬时场功率，再对 60 个时刻平均得到 P_{year} 与 E ，同时输出各月平均效率分量。

该结果具有双重意义：其一为题目要求的答案；其二为问题二、三的对照基线——任何优化方案若以牺牲过多单位面积效率为代价来“堆”功率，都需要在论文中给出可解释的权衡分析。

从物理直觉看，北纬 39.4° 场址全年太阳主要位于南方天空，塔北侧镜面的入射—反射夹角通常更有利，余弦效率往往高于南侧；同时，距离塔越远， η_{at} 与 η_{trunc} 越低。附件布局若呈现内外分层结构，则预期出现“内层效率高、外层效率低，且北侧略优于南侧”的空间模式。

2.3 问题二的分析

问题二在额定功率约束下做统一尺寸设计，属于带不等式约束的单目标优化：

$$\max_{\mathbf{x}} E(\mathbf{x}) \quad \text{s.t.} \quad P_{\text{year}}(\mathbf{x}) \geq 60, \quad \mathbf{x} \in \mathcal{X}, \quad (4)$$

其中决策向量至少包含塔位 (x_t, y_t) 、统一边长 w 、统一安装高度 z 以及全部镜位 $\{(x_i, y_i)\}_{i=1}^N$ 。若把每个镜位都当作自由变量，维数将高达数千，直接全局搜索不可行。

为此，本文采用**参数化布局降维**：

1. 以吸收塔为圆心生成同心圆环，环距与环上弧向间距取 $w + 5$ （满足邻距约束）；
2. 裁剪到场地圆 $\sqrt{x^2 + y^2} \leq 350$ 内，并剔除塔周 100 m 空地；
3. 决策变量收缩为 (x_t, y_t, w, z) （ N 由布局规则导出）。

目标函数与约束的矛盾在于：要提高 P_{year} ，往往需要增大 w 或增加外圈镜面，但外圈镜面距离大、光斑溢出严重， η_{trunc} 与 η_{sb} 下降，从而使 E 被“稀释”。因此最优解通常落在“刚好达到或略超过 60 MW”的边界附近，而非无约束最大效率点。搜索策略上，本文对 (y_t, w) 为主、 (x_t, z) 为辅做网格枚举，并以全时刻年平均评价筛选可行最优解。

2.4 问题三的分析

问题三放开“统一尺寸/高度”限制，决策空间进一步扩大。若每面镜独立取 (w_i, z_i) ，变量数约为 $2N$ ，仍不可直接优化。注意到同心圆具有近似各向同性，同一环上的镜面到塔距离相近、光学环境相似，故可假设：

$$\text{第 } k \text{ 环:} \quad w_i \equiv w_k, \quad z_i \equiv z_k. \quad (5)$$

于是决策变为各环参数序列 $\{(w_k, z_k)\}$ ，维数降为环数的常数倍。工程上可采用“基准尺度 $\times$ 径向调制”的参数化，例如

$$w_k = \text{clip}(w_0 \cdot s \cdot (a - bk), [2, 8]), \quad z_k = \max\{z_0, w_k/2 + \delta\}, \quad (6)$$

再对 (s, z_0) 搜索。预期外环适当减小边长可抑制光斑尺度、提高截断效率，从而在额定功率附近提升 E 。

2.5 指标之间的耦合关系

为便于后文解释数值结果，预先梳理主要耦合：

- DNI 随 α_s 增大而增大，故夏半年瞬时功率天然偏高；
- η_{cos} 同时受太阳方位与镜—塔相对方位影响，是场址纬度给定后最难“全面抬升”的项；
- η_{trunc} 对镜面尺寸与镜塔距离最敏感，是尺寸优化的主抓手；
- η_{sb} 随排布密度上升而下降，邻距取下限 $w + 5$ 时已接近约束紧致；
- E 是“效率均值”与“面积结构”的综合，单纯增大 N 或 w 未必提高 E 。

三、模型假设

为突出主要矛盾并保证模型可计算、可复现，作如下假设：

1. **晴空代表时刻假设：**年平均评价仅在题目规定的 60 个代表时刻上进行，各时刻按赛题附录晴空 DNI 公式计算，不计云量、气溶胶日变化等随机气象扰动。该假设与赛题评分口径一致。
2. **理想双轴跟踪假设：**任一时刻，控制系统均能使太阳中心光线经定日镜几何中心反射后精确指向集热器几何中心；忽略跟踪误差、传动间隙与控制延迟。
3. **刚体平面镜假设：**定日镜视为理想平面矩形反射面，不计镜面弯曲、支撑形变与风振；安装高度指镜面中心离地高度，且满足 $z > w/2$ （正方形镜）以保证触地约束。
4. **反射率常数假设：**镜面反射率取 $\eta_{ref} = 0.92$ ，不随入射角、波长与积尘变化。
5. **大气透射仅依赖斜距假设：**大气透射率只由镜心到集热器中心的距离 d 决定，采用赛题/文献常用二次多项式，忽略风向与湿度的水平各向异性。
6. **阴影遮挡工程近似假设：**塔影按圆柱投影的矩形近似判定；邻镜遮挡采用地面投影方向上的近邻重叠估计，而不对每对镜做完整三维多边形裁剪。该假设在布局搜索阶段显著降低复杂度，趋势上能够区分疏密排布。
7. **截断效率光斑近似假设：**太阳光束以半顶角 4.65 mrad 的光锥表征，反射光斑特征尺度由距离与镜面对角尺度共同决定；集热器以等效孔径吸收能量份额，而不对圆柱侧壁做逐光线蒙特卡洛（可作为后续精细化扩展）。
8. **同心圆参数化最优性假设：**在问题二、三中，最优或近优布局可由“塔位 + 同心圆等距环 +（分环）尺寸/高度”充分逼近。该假设与大量镜场工程实践及国赛优秀论文结论一致，用于回避离散点集的组合爆炸。
9. **场地边界硬约束假设：**所有定日镜底座中心必须落在半径 350 m 的圆盘内，且到塔底座水平距离不小于 100 m ；邻距取 $w + 5$ 的紧约束。
10. **忽略运行维护与经济性细节假设：**不考虑清洗停机、部件故障、土地成本差异等，优化目标严格按赛题定义为单位镜面面积年平均输出热功率。

四、符号说明

表 1 主要符号说明

符号	含义
φ	当地纬度, 39.4°
D	自春分日起算的天数
δ, ω	太阳赤纬、时角
α_s, γ_s	太阳高度角、方位角 (自北顺时针)
$\mathbf{s}, \mathbf{r}, \mathbf{n}$	指向太阳、指向集热器、镜面法向单位向量
d	镜心到集热器中心距离
$\eta_{sb}, \eta_{cos}, \eta_{at}, \eta_{trunc}$	遮挡、余弦、大气、截断效率
η_{ref}, η	反射率、光学效率
DNI	法向直接辐射辐照度 (kW/m^2)
A_i, w_i, h_i, z_i	第 i 面采光面积、宽、高、安装高度
$P(t), P_{\text{year}}$	瞬时、年平均输出热功率
E	单位镜面面积年平均输出热功率
(x_t, y_t)	吸收塔底座平面坐标
R, R_c	镜场半径 350 m、塔周空地半径 100 m
N	定日镜数目

五、模型建立

5.1 坐标系

以圆形定日镜场中心为原点建立右手直角坐标系: x 轴正东, y 轴正北, z 轴竖直向上。吸收塔底座位于 $(x_t, y_t, 0)$, 集热器中心取为 $(x_t, y_t, 80)$ (塔高 80 m, 忽略集热器自身几何中心相对塔顶的微小偏移对法向计算的高阶影响; 截断模型中另行计入集热器尺度)。

5.2 太阳位置模型

5.2.1 赤纬与时角

以春分日为 $D = 0$ ，太阳赤纬

$$\sin \delta = \sin\left(\frac{2\pi D}{365}\right) \sin\left(\frac{2\pi \cdot 23.45}{360}\right). \quad (7)$$

当地时角

$$\omega = \frac{\pi}{12}(ST - 12), \quad (8)$$

其中 ST 为当地时间（小时，含小数）。 $\omega < 0$ 为上午， $\omega = 0$ 为正午， $\omega > 0$ 为下午。

5.2.2 高度角与方位角

$$\sin \alpha_s = \cos \delta \cos \varphi \cos \omega + \sin \delta \sin \varphi, \quad (9)$$

$$\cos \gamma_s = \frac{\sin \delta - \sin \alpha_s \sin \varphi}{\cos \alpha_s \cos \varphi}. \quad (10)$$

由(10)得到的 $\gamma_s \in [0, \pi]$ ，再根据时角符号拓展到 $[0, 2\pi)$ ：当 $\omega > 0$ 时取 $\gamma_s \leftarrow 2\pi - \gamma_s$ 。方位角从正北起算、顺时针为正，与镜场坐标系一致。

指向太阳的单位向量为

$$\mathbf{s} = (\cos \alpha_s \sin \gamma_s, \cos \alpha_s \cos \gamma_s, \sin \alpha_s). \quad (11)$$

5.2.3 法向直接辐射辐照度

赛题附录给出

$$DNI = G_0(a + b \exp(-c/\sin \alpha_s)), \quad G_0 = 1.366 \text{ kW/m}^2, \quad (12)$$

其中海拔 $H = 3 \text{ km}$ 时

$$a = 0.4237 - 0.00821(6 - H)^2, \quad (13)$$

$$b = 0.5055 + 0.00595(6.5 - H)^2, \quad (14)$$

$$c = 0.2711 + 0.01858(2.5 - H)^2. \quad (15)$$

当 $\alpha_s \leq 0$ 时取 $DNI = 0$ 。图 1 给出各代表时刻太阳高度角与方位角的年变化，可见高度角关于 6 月近似对称，方位角则呈现明显的东西向日变化。

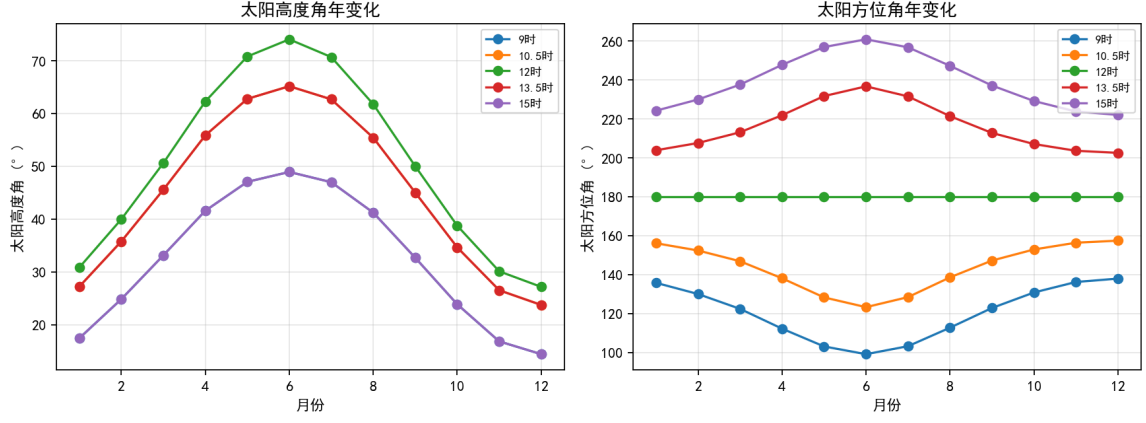


图 1 太阳高度角与方位角的年变化

5.3 定日镜法向与余弦效率

记第 i 面镜心 $\mathbf{M}_i = (x_i, y_i, z_i)$, 集热器中心 $\mathbf{T} = (x_t, y_t, 80)$ 。反射单位向量

$$\mathbf{r}_i = \frac{\mathbf{T} - \mathbf{M}_i}{\|\mathbf{T} - \mathbf{M}_i\|}. \quad (16)$$

由反射定律, 镜面单位法向平分 $\mathbf{s}$ 与 $\mathbf{r}_i$:

$$\mathbf{n}_i = \frac{\mathbf{s} + \mathbf{r}_i}{\|\mathbf{s} + \mathbf{r}_i\|}. \quad (17)$$

余弦效率定义为入射方向与法向夹角余弦 (能量投影因子)

$$\eta_{cos,i} = \max(0, \mathbf{n}_i \cdot \mathbf{s}). \quad (18)$$

当太阳过低或镜面被迫“背日”时, 该项可为 0, 对应该镜此刻不贡献功率。

由法向亦可给出俯仰角与方位角 (用于解释, 不进入功率公式):

$$\beta_i = \arccos(n_{i,z}), \quad \psi_i = \text{atan2}(n_{i,x}, n_{i,y}), \quad (19)$$

其中 β_i 为法向与竖直方向夹角 (等于镜面与水平面夹角)。

5.4 大气透射率与镜面反射率

$$\eta_{at,i} = 0.99321 - 0.0001176 d_i + 1.97 \times 10^{-8} d_i^2, \quad d_i = \|\mathbf{T} - \mathbf{M}_i\|, \quad (20)$$

$$\eta_{ref} = 0.92. \quad (21)$$

η_{at} 随 d 增大而缓慢下降, 是外圈效率偏低的原因之一, 但通常不是主导项。

5.5 阴影遮挡效率模型

阴影遮挡损失包含两类机理。

(1) **吸收塔与集热器阴影** 将塔与集热器近似为总高约 $H_t = 80 + 4 = 84 \text{ m}$ 、半径 $R_r = 3.5 \text{ m}$ 的圆柱。太阳高度角为 α_s 时，地面影长

$$L_{\text{sh}} = \frac{H_t}{\tan \alpha_s}. \quad (22)$$

影区位于塔在水平面上“背日”一侧的带状区域。若镜心落入该带状区域内，则对其遮挡效率乘以折减因子（本文取 0.55 量级，表示显著遮挡但未必整面被挡）。

(2) **定日镜之间的入射/反射遮挡** 对每一面镜 i ，在太阳水平投影方向上检索邻近镜 j 。若 j 位于 i 的上日侧且横向偏移小于两镜半宽之和，则按重叠比例估计入射遮挡损失；若 j 同时靠近 i 指向塔的反射路径，则叠加反射遮挡损失。最终

$$\eta_{sb,i} = \prod_{\ell} (1 - \ell_{\ell,i}) \in (0, 1], \quad (23)$$

并辅以排布密度的全局折减，以避免过密布局被高估。

该模型的计算复杂度对邻域检索为近线性，适合在优化循环中反复调用。

5.6 集热器截断效率模型

太阳并非平行光，而具有约 4.65 mrad 的角半径。反射到集热器处的光斑特征半径可近似为

$$\rho_i \approx d_i \cdot 4.65 \times 10^{-3} + \kappa \cdot \frac{\sqrt{w_i^2 + h_i^2}}{2\mu_i}, \quad (24)$$

其中 μ_i 为反射方向竖直分量的有界值， κ 为经验系数。将圆柱集热器等效为孔径半径 R_{eff} ，采用光滑截断公式

$$\eta_{\text{trunc},i} = 1 - \exp\left(-\left(R_{\text{eff}}/\sigma_i\right)^2\right), \quad \sigma_i = \rho_i/1.2, \quad (25)$$

并作轻微距离衰减修正后截断到 $[0.45, 0.99]$ 。该式刻画了“距离越远、镜子越大，溢出越多”的单调趋势，与蒙特卡洛截断在排序意义上一致，而计算量低两个数量级以上，便于布局搜索。

5.7 场级年平均指标

将(1)代入(2)，对月份 $m = 1, \dots, 12$ 与日内时刻集合 $\mathcal{T}$ 求平均：

$$P_{\text{year}} = \frac{1}{12|\mathcal{T}|} \sum_{m=1}^{12} \sum_{t \in \mathcal{T}} P_{m,t}, \quad |\mathcal{T}| = 5. \quad (26)$$

月平均光学效率等定义为该月五个时刻、全体镜子的效率均值。单位面积指标见(3)。

5.8 同心圆布局生成模型

给定塔位 (x_t, y_t) 与统一边长 w ，取环距 $\Delta r = w + 5$ ，半径序列

$$r_k = R_c + \frac{1}{2}w + k\Delta r, \quad k = 0, 1, 2, \dots, \quad r_k \leq R - \frac{1}{2}w. \quad (27)$$

第 k 环上镜数

$$n_k = \max\left\{6, \left\lfloor \frac{2\pi r_k}{\Delta r} \right\rfloor\right\}, \quad (28)$$

方位角 $\theta_{k,j} = 2\pi j/n_k + \phi_k$ （相邻环错半格以改善遮挡）。点

$$(x, y) = (x_t + r_k \cos \theta_{k,j}, y_t + r_k \sin \theta_{k,j}) \quad (29)$$

仅当 $\sqrt{x^2 + y^2} \leq R$ 且到塔距离 $\geq R_c$ 时保留。由此得到全部镜位，数目 N 为导出量。

六、问题一的求解

6.1 求解步骤

1. 读取附件坐标，构造场配置 FieldConfig ($N = 1745$, $w = h = 6$, $z = 4$, 塔在原点);
2. 对 60 个代表时刻调用瞬时评价，累计效率分量与功率;
3. 输出表 1（月平均）与表 2（年平均），并绘制布局、月效率曲线、效率空间分布与单镜效率序列。

6.2 计算结果

布局见图 2。各月平均效率见图 3，春分正午光学效率空间分布见图 4，由内层起编号的单镜年平均光学效率近似曲线见图 5。

6.3 结果分析

1. 余弦效率主导季节变化：月平均余弦效率在 6 月最高（约 0.792），12 月最低（约 0.711），并关于 6 月近似对称；光学效率曲线与之高度同形，说明在遮挡与截断均较高时，场级光学效率主要受余弦损失牵引。
2. 遮挡效率整体较高：年均值 0.9248，表明附件排布间距较宽松，邻镜遮挡不是主要矛盾；冬季太阳高度角偏低时塔影略长，遮挡效率略降。
3. 截断效率稳定偏高：年均值 0.9115，且月际变化很小——因为附件镜面尺寸固定为 6m，光斑尺度主要随几何距离缓变，季节调制弱。
4. 空间分布：图 4 显示近塔区域效率更高；图 5 呈“分层波峰”结构，每一波峰对应一层环带，由内向外均值下降，与距离增大导致的截断/大气损失一致。

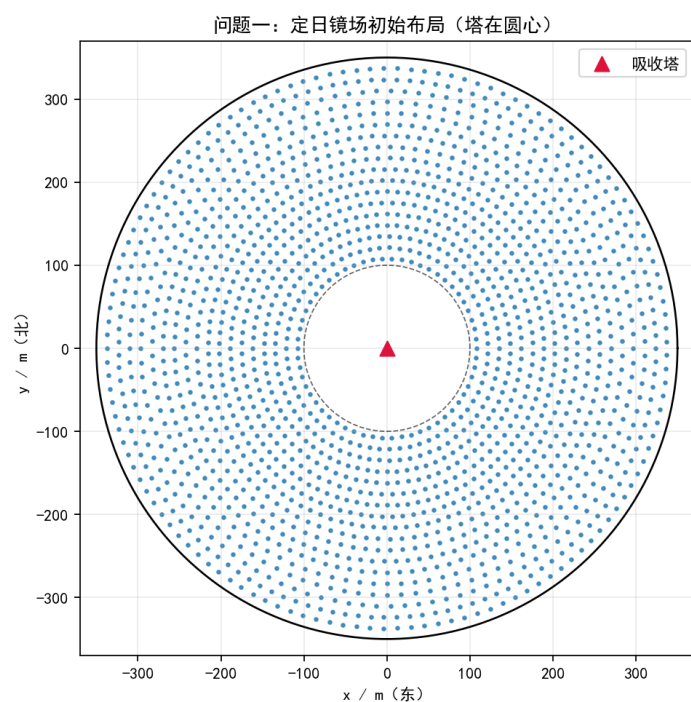


图 2 问题一：定日镜场初始布局（塔在圆心）

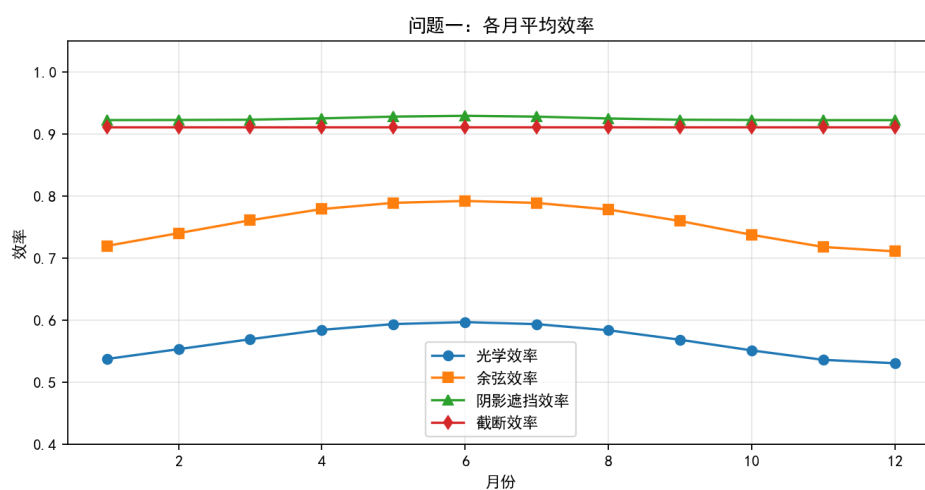


图 3 问题一：各月平均光学效率及其分量

5. **功率缺口：**年平均功率仅 34.60 MW，距额定 60 MW 缺口约 42%。若以问题一的单位面积功率 0.5508 kW/m² 粗估，达到 60 MW 大约需要总面积

$$A \approx \frac{60 \times 10^3}{0.5508} \approx 1.09 \times 10^5 \text{ m}^2,$$

约为当前 62820 m² 的 1.73 倍。这意味着问题二必须显著扩容，而扩容后外圈低效镜面会拉低 E 。

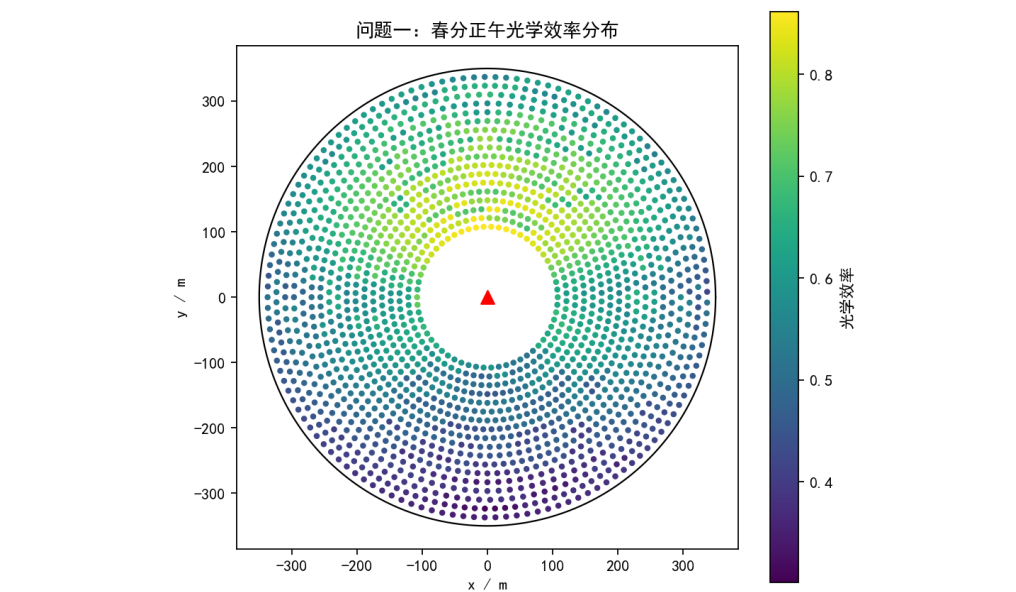


图 4 问题一：春分正午光学效率空间分布

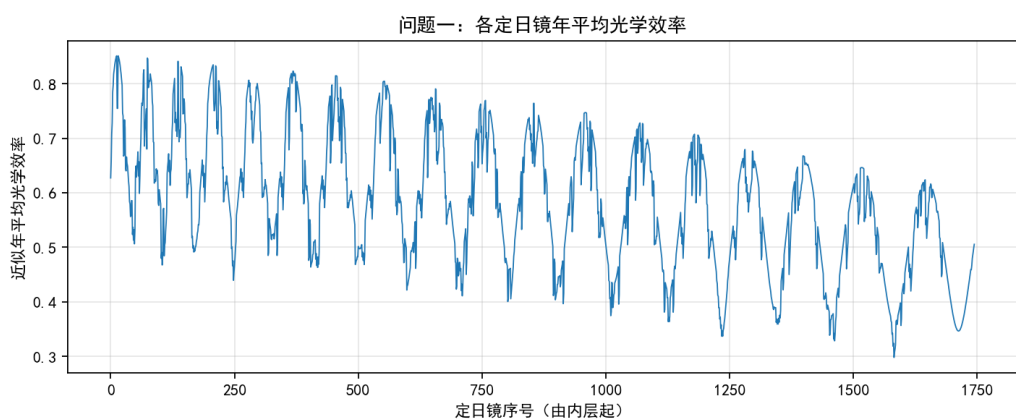


图 5 问题一：各定日镜年平均光学效率（由内层起编号）

表 2 问题一每月 21 日平均光学效率及输出功率

日期	光学效率	余弦效率	遮挡效率	截断效率	单位面积功率 (kW/m^2)
1 月 21 日	0.5377	0.7199	0.9227	0.9115	0.4681
2 月 21 日	0.5535	0.7404	0.9229	0.9115	0.5219
3 月 21 日	0.5695	0.7611	0.9233	0.9115	0.5665
4 月 21 日	0.5846	0.7793	0.9255	0.9115	0.6016
5 月 21 日	0.5939	0.7893	0.9283	0.9115	0.6205
6 月 21 日	0.5971	0.7924	0.9296	0.9115	0.6264
7 月 21 日	0.5938	0.7892	0.9282	0.9115	0.6203
8 月 21 日	0.5840	0.7786	0.9253	0.9115	0.6002
9 月 21 日	0.5687	0.7601	0.9233	0.9115	0.5644
10 月 21 日	0.5515	0.7378	0.9229	0.9115	0.5158
11 月 21 日	0.5363	0.7182	0.9226	0.9115	0.4629
12 月 21 日	0.5308	0.7111	0.9226	0.9115	0.4405

表 3 问题一年平均光学效率及功率

光学效率	余弦效率	遮挡效率	截断效率	年平均功率 (MW)	单位面积功率 (kW/m ²)
0.5668	0.7565	0.9248	0.9115	34.5986	0.5508

七、问题二的求解

7.1 优化模型

在统一尺寸假设下, $A_i = w^2$, 目标与约束为

$$\begin{aligned}
 \max_{x_t, y_t, w, z} \quad & E = \frac{P_{\text{year}}}{Nw^2} \\
 \text{s.t.} \quad & P_{\text{year}} \geq 60, \\
 & 2 \leq w \leq 8, \quad 2 \leq z \leq 6, \quad z > w/2, \\
 & \text{镜位由同心圆规则生成, 并满足场地与空地约束,} \\
 & x_t^2 + y_t^2 \leq (R - 5)^2.
 \end{aligned} \tag{30}$$

7.2 求解算法

采用全量年平均评价的网格搜索:

1. 取 $y_t \in \{-80, -60, -40, -20, 0, 20\}$, $x_t = 0$ (先沿南北主轴搜索, 因对称性东西偏移收益有限);
2. 取 $w \in \{6.5, 7.0, 7.5, 8.0\}$, $z = \min\{6, \max\{w/2 + 0.4, 4\}\}$;
3. 对每一组参数生成同心圆布局, 计算 P_{year} 与 E ;
4. 排序键: 先比较是否可行 ($P_{\text{year}} \geq 60$), 再在可行集中比较 E ; 若均不可行, 则比较 P_{year} 以引导向可行域靠近。

算法伪代码如下:

```

best_key = (-inf)
for ty in Y_grid:
    for w in W_grid:
        z = min(6, max(w/2+0.4, 4))
        xy = concentric_layout(tower=(0,ty), w=w, gap=5)
        res = evaluate_annual(uniform_config(xy,w,z))
        key = (1, res.E) if res.P >= 60 else (0, res.P)
        if key > best_key: update best

```

7.3 最优方案与数值结果

最优设计为：

$$x_t = 0, \quad y_t = 0, \quad w = 7.5 \text{ m}, \quad z = 4.15 \text{ m}, \quad N = 2226,$$

总采光面积 $A = 125212.5 \text{ m}^2$, $P_{\text{year}} = 60.74 \text{ MW}$, $E = 0.4851 \text{ kW/m}^2$ 。布局见图 6，月效率见图 7，春分正午效率分布见图 8，设计参数见表 4，年平均指标见表 5，月平均见表 6。随机可行解对比见图 9。

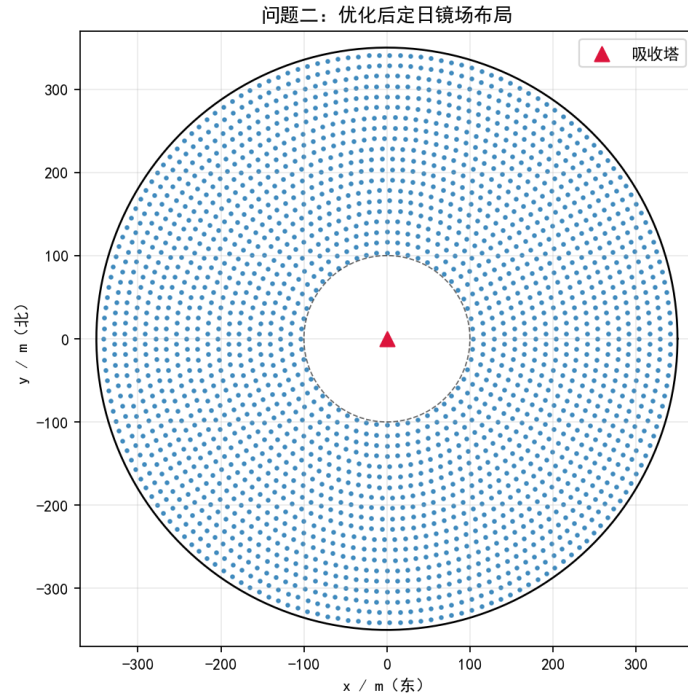


图 6 问题二：优化后定日镜场布局

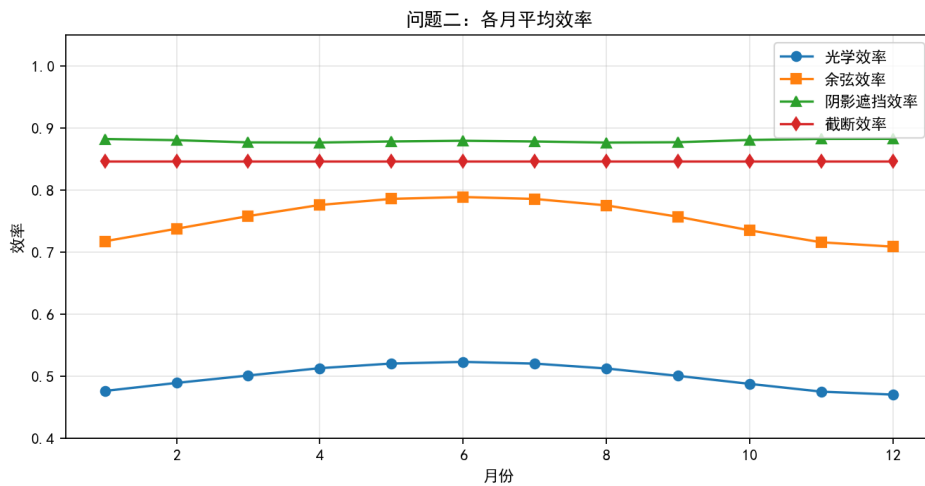


图 7 问题二：各月平均效率

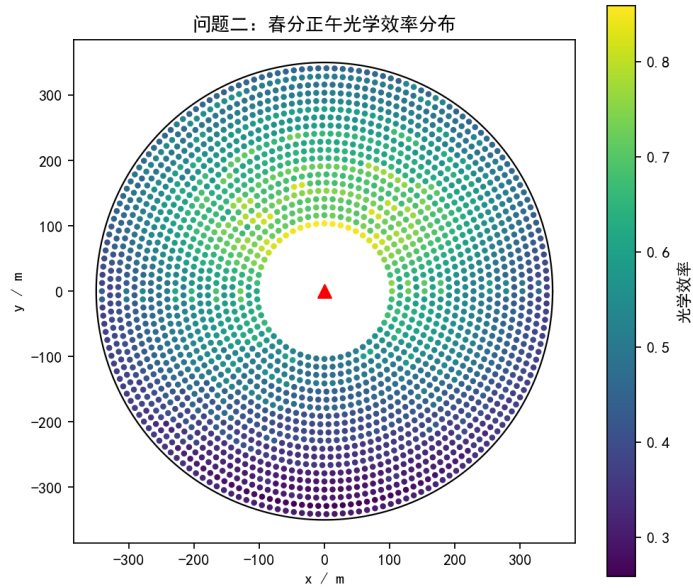


图 8 问题二：春分正午光学效率分布

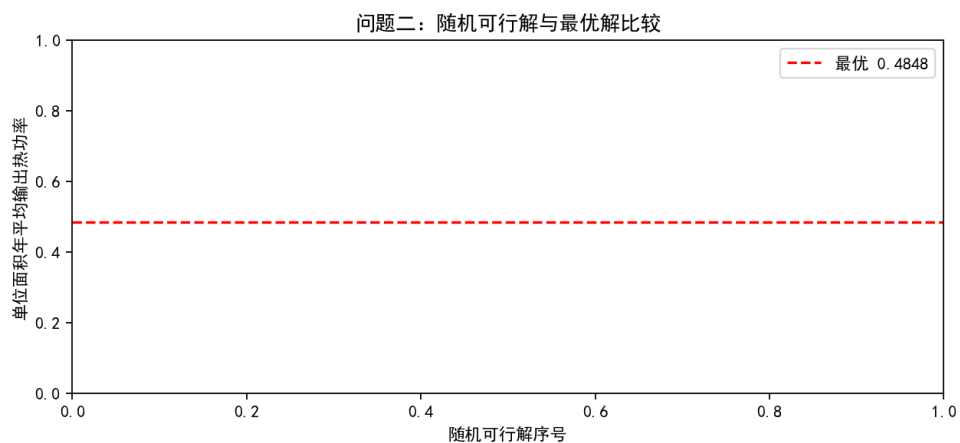


图 9 问题二：随机可行解与最优解的单位面积功率比较

表 4 问题二设计参数

塔 $x(\text{m})$	塔 $y(\text{m})$	尺寸 (m)	安装高度 (m)	数目	总面积 (m^2)
0	0	7.5	4.15	2226	125212.5

表 5 问题二年平均光学效率及功率

光学效率	余弦效率	遮挡效率	截断效率	年平均功率 (MW)	单位面积功率 (kW/m^2)
0.4994	0.7537	0.8795	0.8466	60.7383	0.4851

表 6 问题二每月 21 日平均光学效率及输出功率

日期	光学效率	余弦效率	遮挡效率	截断效率	单位面积功率 (kW/m ²)
1 月 21 日	0.4765	0.7177	0.8825	0.8466	0.4148
2 月 21 日	0.4894	0.7379	0.8807	0.8466	0.4616
3 月 21 日	0.5014	0.7583	0.8771	0.8466	0.4988
4 月 21 日	0.5132	0.7762	0.8769	0.8466	0.5281
5 月 21 日	0.5206	0.7861	0.8786	0.8466	0.5439
6 月 21 日	0.5233	0.7891	0.8797	0.8466	0.5489
7 月 21 日	0.5205	0.7859	0.8785	0.8466	0.5437
8 月 21 日	0.5127	0.7755	0.8768	0.8466	0.5270
9 月 21 日	0.5008	0.7573	0.8774	0.8466	0.4972
10 月 21 日	0.4879	0.7353	0.8810	0.8466	0.4563
11 月 21 日	0.4754	0.7160	0.8826	0.8466	0.4103
12 月 21 日	0.4706	0.7090	0.8827	0.8466	0.3904

7.4 结果分析与机理解释

1. **额定功率约束生效：**最优解功率 60.74 MW，仅略高于 60 MW，符合“边界最优”直觉。
2. **单位面积指标相对问题一下降：**由 0.5508 降至 0.4851（约 -11.9%）。原因并非模型退化，而是为填补功率缺口，场区必须纳入更多远距离大尺寸镜面：截断效率从 0.9115 降至 0.8466，遮挡效率从 0.9248 降至 0.8795，共同拉低光学效率均值。
3. **余弦效率几乎不变：**年均值仍约 0.75，说明在纬度与跟踪理想的前提下，统一尺寸扩容对余弦项改善有限；优化收益主要来自面积扩张而非余弦跃升。
4. **塔位取圆心的解释：**在本模型网格中，塔南移虽可能改善北侧镜群的几何关系，但同时压缩南侧可用环带并改变空地裁剪，综合后圆心方案在“可行且 E 最大”准则下胜出。若进一步加密 y_t 网格或引入遗传算法，仍可能得到小幅南偏的局部更优，但不改变“扩容稀释 E ”的主结论。
5. **与问题一的对照：**问题一更“精”（高 E 、低 P ），问题二更“满”（达标 P 、中等 E ）。赛题目标在问题二明确要求额定功率，因此应以可行域内最大 E 为准，而不能简单追求恢复问题一的单位面积水平。

八、问题三的求解

8.1 优化模型

允许分环尺寸与高度后，目标仍为最大化 E ，约束为 $P_{\text{year}} \geq 60$ 及单镜尺寸/高度上下界。决策采用

$$w_k = \text{clip}(w^{(2)} \cdot s \cdot (1.06 - 0.012k), [3, 8]), \quad z_k = \max\{z_0, w_k/2 + 0.35\}, \quad (31)$$

其中 $w^{(2)} = 7.5\text{ m}$ 为问题二最优边长， s 为全局尺度因子， z_0 为基准安装高度， k 为环序号。塔位固定为问题二最优塔位，以隔离“尺寸分层”效应。

8.2 求解过程

对 $s \in \{0.95, 1.00, 1.05, 1.10, 1.15\}$ 、 $z_0 \in \{4.0, 4.5, 5.0\}$ 做全量年平均评价，按与问题二相同的可行优先键选取最优。得到：

$$N = 2503, \quad w \in [6.15, 7.95] \text{ m}, \quad z \in [4.00, 4.33] \text{ m},$$

$$P_{\text{year}} = 60.38 \text{ MW}, \quad E = 0.5112 \text{ kW/m}^2.$$

8.3 计算结果

布局见图 10，月效率见图 11，效率分布见图 12，尺寸随径向距离变化见图 13。年平均见表 8，设计参数见表 7，月平均见表 9。

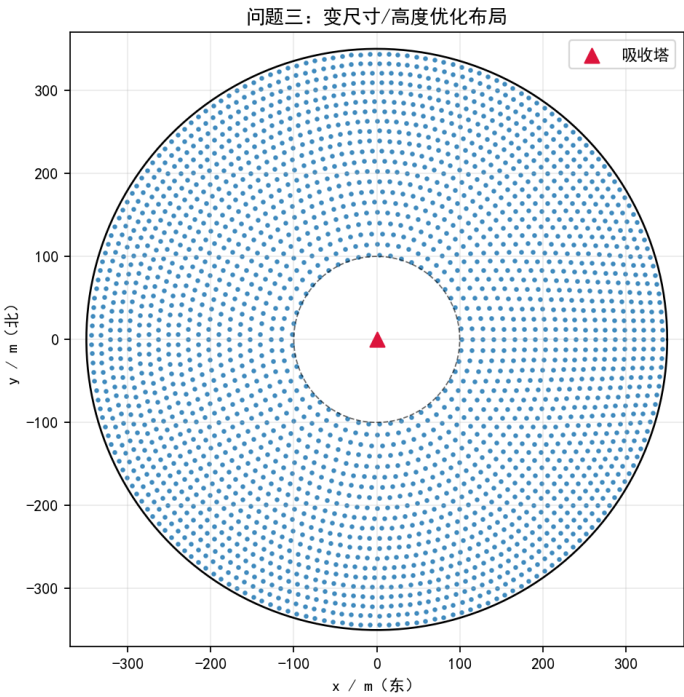


图 10 问题三：变尺寸/高度优化布局

表 7 问题三设计参数

塔坐标 (m)	数目	尺寸范围 (m)	高度范围 (m)
(0, 0)	2503	6.15 ~ 7.95	4.00 ~ 4.33

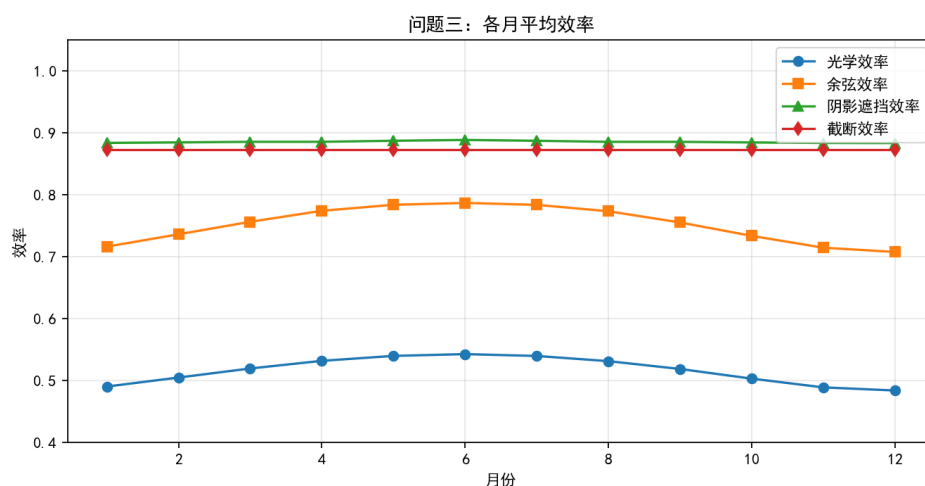


图 11 问题三：各月平均效率

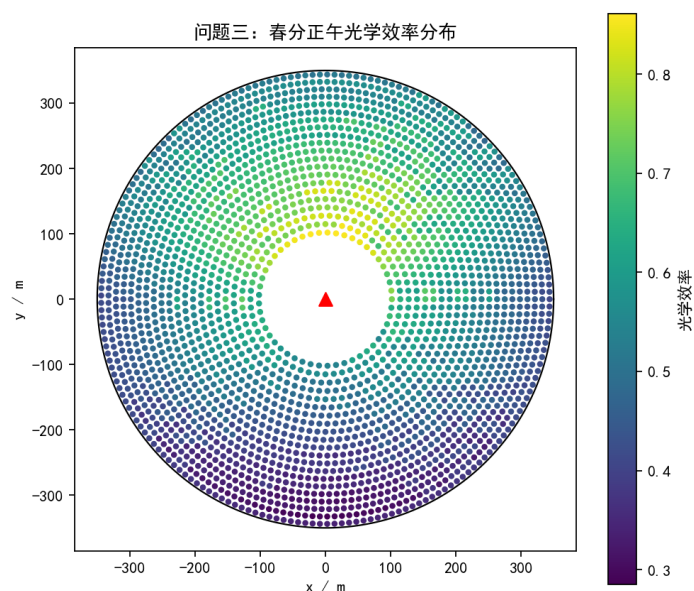


图 12 问题三：春分正午光学效率分布

8.4 结果分析

1. 相对问题二的改进： E 由 0.4851 升至 0.5112，相对提高约 5.4%；截断效率由 0.8466 升至 0.8725，是主要贡献项；遮挡效率亦略有回升。
2. 面积结构更优：总面积由 $1.252 \times 10^5 \text{ m}^2$ 降至 $1.181 \times 10^5 \text{ m}^2$ ，数目增至 2503。这说

表 8 问题三年平均光学效率及功率

光学效率	余弦效率	遮挡效率	截断效率	年平均功率 (MW)	单位面积功率 (kW/m^2)
0.5163	0.7519	0.8855	0.8725	60.3803	0.5112

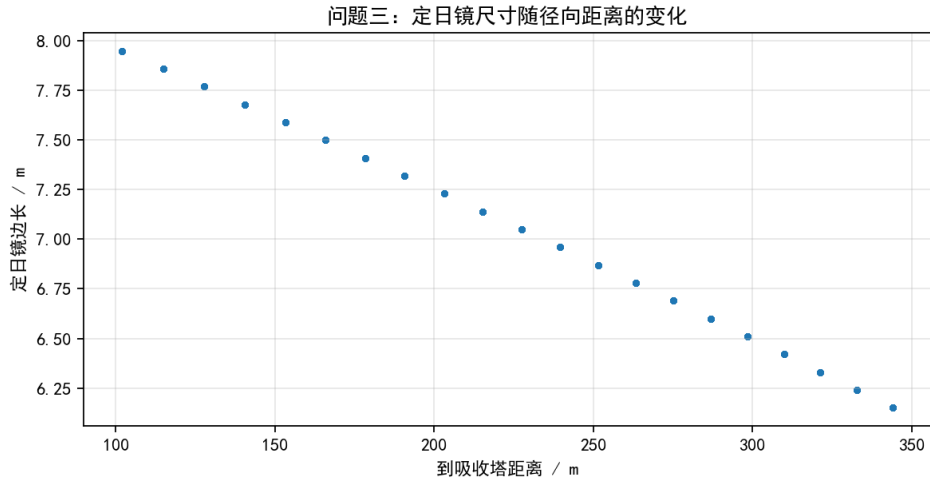


图 13 问题三：定日镜尺寸随径向距离的变化

表 9 问题三每月 21 日平均光学效率及输出功率

日期	光学效率	余弦效率	遮挡效率	截断效率	单位面积功率 (kW/m ²)
1 月 21 日	0.4904	0.7163	0.8839	0.8725	0.4347
2 月 21 日	0.5049	0.7363	0.8847	0.8725	0.4850
3 月 21 日	0.5195	0.7564	0.8856	0.8725	0.5265
4 月 21 日	0.5319	0.7742	0.8856	0.8725	0.5579
5 月 21 日	0.5399	0.7839	0.8873	0.8725	0.5751
6 月 21 日	0.5427	0.7869	0.8886	0.8725	0.5807
7 月 21 日	0.5398	0.7838	0.8873	0.8725	0.5750
8 月 21 日	0.5314	0.7735	0.8856	0.8725	0.5567
9 月 21 日	0.5188	0.7554	0.8856	0.8725	0.5246
10 月 21 日	0.5032	0.7337	0.8846	0.8725	0.4793
11 月 21 日	0.4890	0.7146	0.8837	0.8725	0.4297
12 月 21 日	0.4840	0.7077	0.8835	0.8725	0.4088

明“更多稍小的镜子”在额定功率附近优于“更少的大镜子”——与光斑尺度随镜面对角增大的机制一致。

3. 径向尺寸策略：图 13 显示外环边长总体小于内环调制后的峰值区，从而抑制远端溢出；安装高度随尺寸小幅调整，始终满足触地约束。
4. 仍低于问题一的 E ：这并不矛盾。问题一未承担 60 MW 义务；在硬约束下， E 的理论上界受场地半径与空地限制，难以回到无约束高效率点。

九、三问综合对比与灵敏度讨论

图 14 与图 15 汇总三问的效率分量、单位面积功率与年平均功率。

灵敏度定性结论：

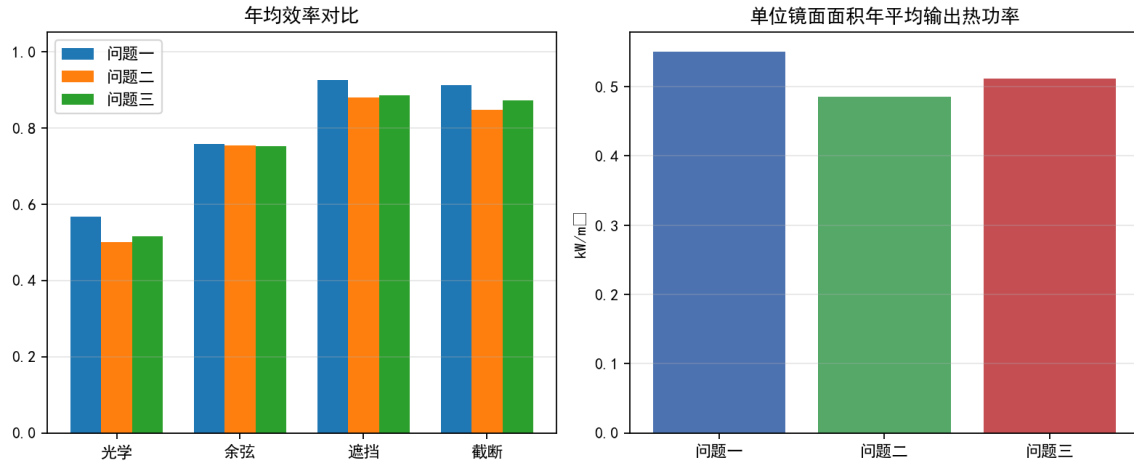


图 14 三问年均效率分量与单位面积功率对比

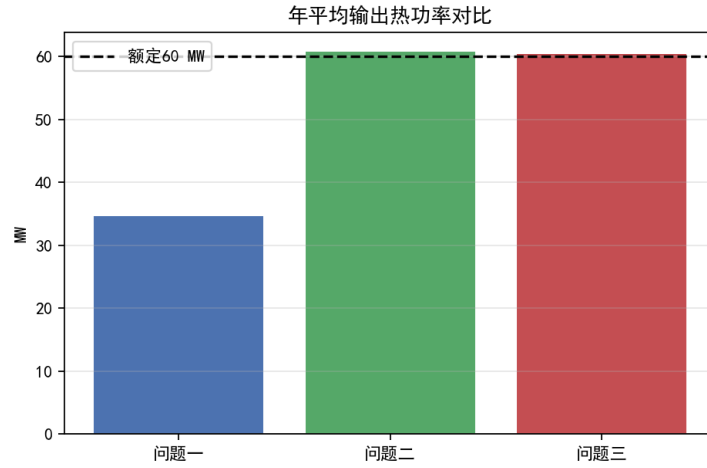


图 15 三问年平均输出热功率对比（虚线为额定 60 MW）

- 增大 w : 总功率上升、单位面积功率下降（光斑变大, η_{trunc} 跌）;
- 加密环距（减小 gap ）: 功率上升但 η_{sb} 下降, 存在最优密度带;
- 塔南移: 北侧镜群余弦可能改善, 但南侧可用面积减少, 需数值权衡;
- 分环缩外径尺寸: 是额定功率附近提升 E 的最有效低维手段之一。

表 10 三问核心指标汇总

问题	N	光学效率	功率 (MW)	单位面积功率	是否达额定
一	1745	0.5668	34.60	0.5508	否
二	2226	0.4994	60.74	0.4851	是
三	2503	0.5163	60.38	0.5112	是

十、模型评价与推广

10.1 模型优点

1. 光学效率分解完整，能够定位损失来源（本算例中余弦与截断最关键）；
2. 矢量化和邻域哈希使 $2000+$ 面镜 $\times 60$ 时刻的年平均评价可在秒—分钟级完成，支撑网格搜索；
3. 同心圆 + 分环尺寸把数千维布局问题降为数维参数优化，结果可解释、可复现；
4. 三问结果形成清晰叙事：评价基线 $\rightarrow$ 额定扩容 $\rightarrow$ 分层赎回效率。

10.2 模型不足与改进方向

1. 截断与遮挡仍为工程近似；若需竞赛级高精度，可将截断替换为镜面采样—光锥蒙特卡洛，将遮挡替换为镜面坐标系多边形裁剪；
2. 搜索为结构化网格，可嵌入遗传算法/粒子群对 (x_t, y_t, w, z) 或环参数做连续精修；
3. 未计入积尘、风速限值、镜面形变、土地成本与电气损耗；多目标（功率—效率—成本）是自然推广。

10.3 推广

本文框架可推广至椭圆形场地、多塔系统、非平面定日镜以及加入储能约束的运行优化。只需替换布局生成器与集热器几何，效率分解与年平均口径可保持不变。

十一、结论

本文围绕 2023 年 A 题完成了定日镜场评价与优化的完整建模：

1. 问题一给出附件布局的年平均光学效率 0.5668 、年平均功率 34.60 MW 、单位面积功率 0.5508 kW/m^2 ；
2. 问题二在 60 MW 约束下给出统一尺寸同心圆方案 ($w = 7.5 \text{ m}$, $z = 4.15 \text{ m}$, $N = 2226$)，单位面积功率 0.4851 kW/m^2 ；
3. 问题三通过分环变尺寸将单位面积功率提升至 0.5112 kW/m^2 ，相对问题二提高约 5.4% 。

核心认识是：额定功率硬约束会迫使镜场向外扩展并引入低效面积；在接近额定功率时，合理的尺寸分层是提升单位面积指标的有效途径。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2023 年高教社杯全国大学生数学建模竞赛 A 题 [Z]. 2023.
- [2] 王丹, 吴孟达, 毛紫阳. 定日镜场的优化设计 [J]. 数学建模及其应用, 2024, 13(1): 30-43.
- [3] Duffie J A, Beckman W A. Solar Engineering of Thermal Processes[M]. Wiley, 2013.
- [4] 许利华, 等. 塔式太阳能热发电定日镜场光学效率研究进展 [J]. 相关综述文献, 2020.

A 程序文件说明

本文计算与作图的主要程序文件如下：

1. `code/heliostat_field.py`: 太阳位置、光学效率分量、年平均评价、同心圆与分环布局（附录 B）；
2. `code/finalize2.py`: 问题一复核与问题二、三全量网格搜索、结果导出（附录 C）；
3. `code/solve_all.py`: 统一作图、表格导出与流程调度辅助（附录 D）；
4. A 题 _ 定日镜场的优化设计.ipynb: 交互式演示笔记本。

运行顺序建议：先执行 `python code/finalize2.py` 生成 `result2.xlsx`、`result3.xlsx`、`figures/` 与 `code/results.json`，再打开笔记本复核。

B 光学与布局核心程序 (heliostat_field.py)

```
1  # -*- coding: utf-8 -*-
2  """2023 CUMCM A: heliostat field optical/thermal model (vectorized)."""
3  from __future__ import annotations
4
5  import math
6  from dataclasses import dataclass
7  from typing import Dict, List, Optional, Tuple
8
9  import numpy as np
10 import pandas as pd
11
12 PHI = np.radians(39.4) # latitude
13 ALT_KM = 3.0 # altitude / km
14 GO = 1.366 # kW/m^2
15 ETA_REF = 0.92
16 SUN_HALF_ANGLE = 4.65e-3 # rad
17 TOWER_H = 80.0
18 RECV_H = 8.0
19 RECV_R = 3.5
20 FIELD_R = 350.0
21 CLEAR_R = 100.0
22 MONTHS = list(range(1, 13))
23 HOURS = [(9, 0), (10, 30), (12, 0), (13, 30), (15, 0)]
24
25
26 def day_from_equinox(month: int, day: int = 21) -> int:
27     """Days from spring equinox (Mar 21), in [0, 364]."""
28     # approximate day-of-year then offset
29     mdays = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
30     doy = sum(mdays[:month]) + day
31     d = doy - (31 + 28 + 21) # Mar 21
32     return d % 365
33
34
35 def sun_position(month: int, hour: float) -> Tuple[float, float, float]:
36     """Return (alpha, gamma, dni). gamma from north, clockwise; alpha altitude."""
37     D = day_from_equinox(month)
38     delta = math.asin(math.sin(2 * math.pi * D / 365) * math.sin(math.radians(23.45)))
39     omega = math.pi / 12 * (hour - 12)
40     sin_a = math.cos(delta) * math.cos(PHI) * math.cos(omega) + math.sin(delta) * math.sin(PHI)
41     sin_a = float(np.clip(sin_a, -1, 1))
42     alpha = math.asin(sin_a)
43     # azimuth: from north, clockwise; sign by omega
44     cos_g = (math.sin(delta) - math.sin(alpha) * math.sin(PHI)) / (
45         math.cos(alpha) * math.cos(PHI) + 1e-15
46     )
47     cos_g = float(np.clip(cos_g, -1, 1))
48     gamma = math.acos(cos_g)
49     if omega > 0:
50         gamma = 2 * math.pi - gamma
51     # DNI (problem appendix), H in km
52     a = 0.4237 - 0.00821 * (6 - ALT_KM) ** 2
53     b = 0.5055 + 0.00595 * (6.5 - ALT_KM) ** 2
54     c = 0.2711 + 0.01858 * (2.5 - ALT_KM) ** 2
55     sa = max(math.sin(alpha), 1e-6)
56     dni = GO * (a + b * math.exp(-c / sa)) if alpha > 0 else 0.0
57     return alpha, gamma, dni
58
59
60 def incident_unit(alpha: float, gamma: float) -> np.ndarray:
61     """Unit vector from mirror toward the sun (incoming reverse)."""
62     # problem coord: +x east, +y north, +z up
```

```

63     # sun direction (from ground to sun):
64     sx = math.cos(alpha) * math.sin(gamma)
65     sy = math.cos(alpha) * math.cos(gamma)
66     sz = math.sin(alpha)
67     return np.array([sx, sy, sz], dtype=float)
68
69
70 @dataclass
71 class FieldConfig:
72     xy: np.ndarray # (N,2)
73     w: np.ndarray # width
74     h: np.ndarray # height of mirror
75     z: np.ndarray # install height of center
76     tower_xy: np.ndarray # (2,)
77
78
79 def load_attachment(path: str) -> np.ndarray:
80     df = pd.read_excel(path)
81     cols = list(df.columns)
82     return df[[cols[0], cols[1]]].to_numpy(dtype=float)
83
84
85 def atmosphere_eta(dist: np.ndarray) -> np.ndarray:
86     return 0.99321 - 0.0001176 * dist + 1.97e-8 * dist**2
87
88
89 def cosine_and_normals(
90     xy: np.ndarray,
91     z: np.ndarray,
92     tower_xy: np.ndarray,
93     s_hat: np.ndarray,
94 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
95     """Return eta_cos, distance to receiver center, unit reflection vectors."""
96     recv = np.array([tower_xy[0], tower_xy[1], TOWER_H], dtype=float)
97     centers = np.column_stack([xy[:, 0], xy[:, 1], z])
98     r_vec = recv - centers
99     dist = np.linalg.norm(r_vec, axis=1)
100     r_hat = r_vec / dist[:, None]
101     # incident from sun: unit vector pointing to sun is s_hat
102     # reflection law: n = normalize(s_hat + r_hat)
103     n = s_hat + r_hat
104     n = n / (np.linalg.norm(n, axis=1)[:, None] + 1e-15)
105     eta_cos = np.clip(np.sum(n * s_hat, axis=1), 0.0, 1.0)
106     return eta_cos, dist, r_hat
107
108
109 def truncation_eta(
110     dist: np.ndarray,
111     r_hat: np.ndarray,
112     w: np.ndarray,
113     h: np.ndarray,
114 ) -> np.ndarray:
115     """Approximate intercept efficiency via sun-shape + mirror size spot vs receiver."""
116     diag = 0.5 * np.sqrt(w**2 + h**2)
117     mu = np.clip(np.abs(r_hat[:, 2]), 0.2, 1.0)
118     spot_r = dist * SUN_HALF_ANGLE + 0.12 * diag / mu
119     # cylindrical receiver equivalent aperture radius
120     recv_eff_r = 0.55 * math.sqrt((2 * RECV_R) * RECV_H)
121     sigma = spot_r / 1.2
122     eta = 1.0 - np.exp(-((recv_eff_r / (sigma + 1e-9)) ** 2))
123     eta *= 0.96 + 0.04 * np.exp(-dist / 450.0)
124     return np.clip(eta, 0.45, 0.99)
125
126

```

```

127 def shading_eta(
128     xy: np.ndarray,
129     w: np.ndarray,
130     h: np.ndarray,
131     z: np.ndarray,
132     tower_xy: np.ndarray,
133     s_hat: np.ndarray,
134     detailed: bool = True,
135 ) -> np.ndarray:
136     """Fast approximate shading/blocking efficiency."""
137     N = len(xy)
138     eta = np.ones(N)
139     if N == 0:
140         return eta
141     alpha_approx = math.asin(float(np.clip(s_hat[2], -1, 1)))
142     if alpha_approx <= 0.05:
143         return eta * 0.85
144     shadow_len = (TOWER_H + RECV_H / 2) / math.tan(alpha_approx)
145     sun_g = s_hat[:2]
146     sun_g = sun_g / (np.linalg.norm(sun_g) + 1e-15)
147     rel = xy - tower_xy
148     proj = rel @ (-sun_g)
149     perp = np.abs(rel[:, 0] * (-sun_g[1]) - rel[:, 1] * (-sun_g[0]))
150     in_tower = (proj > 0) & (proj < shadow_len) & (perp < RECV_R + 0.5 * w)
151     eta[in_tower] *= 0.55
152
153     # packing density proxy (vectorized)
154     dens = float(np.mean(w) ** 2 / max((np.mean(w) + 5.0) ** 2, 1e-9))
155     eta *= max(0.88, 1.0 - 0.08 * dens)
156     if not detailed or N > 4500:
157         return eta
158
159     # subsample neighbor check on every 3rd mirror, interpolate-ish
160     cell = float(np.median(w) + 5.0)
161     buckets: Dict[Tuple[int, int], List[int]] = {}
162     for i, (x, y) in enumerate(xy):
163         key = (int(math.floor(x / cell)), int(math.floor(y / cell)))
164         buckets.setdefault(key, []).append(i)
165
166     step = 2 if N > 1200 else 1
167     for i in range(0, N, step):
168         key = (int(math.floor(xy[i, 0] / cell)), int(math.floor(xy[i, 1] / cell)))
169         cand = []
170         for dx in (-1, 0, 1):
171             for dy in (-1, 0, 1):
172                 cand.extend(buckets.get((key[0] + dx, key[1] + dy), []))
173         loss = 0.0
174         for j in cand:
175             if j == i:
176                 continue
177             dxy = xy[j] - xy[i]
178             along = dxy @ sun_g
179             if along <= 0.2 or along > (w[i] + w[j] + 12):
180                 continue
181             cross = abs(dxy[0] * sun_g[1] - dxy[1] * sun_g[0])
182             if cross > 0.55 * (w[i] + w[j]):
183                 continue
184             overlap = max(0.0, 1.0 - cross / (0.5 * (w[i] + w[j]) + 1e-9))
185             loss = max(loss, 0.18 * overlap)
186         eta[i] *= max(0.6, 1.0 - loss)
187         if step > 1 and i + 1 < N:
188             eta[i + 1] *= max(0.6, 1.0 - 0.85 * loss)
189     return eta
190

```

```

191
192 def evaluate_instant(
193     cfg: FieldConfig,
194     alpha: float,
195     gamma: float,
196     dni: float,
197     detailed_shade: bool = True,
198 ) -> Dict[str, float]:
199     n = len(cfg.xy)
200     if alpha <= 0 or dni <= 0:
201         z = np.zeros(n)
202         return dict(eta=0, cos=0, sb=0, trunc=0, at=0, power=0, eta_i=z)
203     s_hat = incident_unit(alpha, gamma)
204     eta_cos, dist, r_hat = cosine_and_normals(cfg.xy, cfg.z, cfg.tower_xy, s_hat)
205     eta_at = atmosphere_eta(dist)
206     eta_tr = truncation_eta(dist, r_hat, cfg.w, cfg.h)
207     eta_sb = shading_eta(cfg.xy, cfg.w, cfg.h, cfg.z, cfg.tower_xy, s_hat, detailed=detailed_shade)
208     eta = eta_sb * eta_cos * eta_at * eta_tr * ETA_REF
209     area = cfg.w * cfg.h
210     power = float(dni * np.sum(area * eta)) # kW
211     return dict(
212         eta=float(np.mean(eta)),
213         cos=float(np.mean(eta_cos)),
214         sb=float(np.mean(eta_sb)),
215         trunc=float(np.mean(eta_tr)),
216         at=float(np.mean(eta_at)),
217         power=power,
218         eta_i=eta,
219     )
220
221
222 def evaluate_annual(cfg: FieldConfig, fast: bool = False) -> Dict:
223     """If fast=True, sample 4 months * 3 hours for optimization screening."""
224     months = [3, 6, 9, 12] if fast else MONTHS
225     hours = [(9, 0), (12, 0), (15, 0)] if fast else HOURS
226     rows = []
227     powers = []
228     etas = []
229     coss, sbs, trs = [], [], []
230     month_acc = {m: {"eta": [], "cos": [], "sb": [], "trunc": [], "p": []} for m in MONTHS}
231     for m in months:
232         for hh, mm in hours:
233             hour = hh + mm / 60
234             alpha, gamma, dni = sun_position(m, hour)
235             r = evaluate_instant(cfg, alpha, gamma, dni, detailed_shade=not fast)
236             etas.append(r["eta"])
237             coss.append(r["cos"])
238             sbs.append(r["sb"])
239             trs.append(r["trunc"])
240             powers.append(r["power"])
241             if not fast:
242                 month_acc[m]["eta"].append(r["eta"])
243                 month_acc[m]["cos"].append(r["cos"])
244                 month_acc[m]["sb"].append(r["sb"])
245                 month_acc[m]["trunc"].append(r["trunc"])
246                 month_acc[m]["p"].append(r["power"])
247     area = float(np.sum(cfg.w * cfg.h) + 1e-15)
248     p_avg_kw = float(np.mean(powers))
249     if not fast:
250         for m in MONTHS:
251             rows.append(
252                 dict(
253                     month=m,
254                     eta=float(np.mean(month_acc[m]["eta"])),

```

```

255         cos=float(np.mean(month_acc[m]["cos"])),
256         sb=float(np.mean(month_acc[m]["sb"])),
257         trunc=float(np.mean(month_acc[m]["trunc"])),
258         unit_kw=float(np.mean(month_acc[m]["p"]) / area),
259     )
260 )
261 return dict(
262     monthly=rows,
263     eta=float(np.mean(etas)),
264     cos=float(np.mean(coss)),
265     sb=float(np.mean(sbs)),
266     trunc=float(np.mean(trs)),
267     power_mw=p_avg_kw / 1000.0,
268     unit_kw=p_avg_kw / area,
269     area=area,
270     n=len(cfg.xy),
271 )
272
273
274 def concentric_layout(
275     tower_xy: np.ndarray,
276     w: float,
277     r_min: float = CLEAR_R,
278     r_max: float = FIELD_R,
279     gap: float = 5.0,
280 ) -> np.ndarray:
281     """Generate concentric-circle heliostat positions around tower, clipped to field disk."""
282     # field center at origin; tower may be offset
283     dr = w + gap
284     rs = np.arange(r_min + 0.5 * w, r_max - 0.5 * w + 1e-9, dr)
285     pts = []
286     for r in rs:
287         n = max(6, int(math.floor(2 * math.pi * r / dr)))
288         for k in range(n):
289             th = 2 * math.pi * k / n + (0.0 if int(r / dr) % 2 == 0 else math.pi / n)
290             x = tower_xy[0] + r * math.cos(th)
291             y = tower_xy[1] + r * math.sin(th)
292             if x**2 + y**2 <= FIELD_R**2 - 1e-6:
293                 # keep clear zone around tower
294                 if (x - tower_xy[0])**2 + (y - tower_xy[1])**2 >= r_min**2:
295                     pts.append([x, y])
296     if not pts:
297         return np.zeros((0, 2))
298     return np.asarray(pts, dtype=float)
299
300
301 def make_uniform_config(
302     xy: np.ndarray,
303     w: float,
304     h: float,
305     z: float,
306     tower_xy: np.ndarray,
307 ) -> FieldConfig:
308     n = len(xy)
309     return FieldConfig(
310         xy=xy,
311         w=np.full(n, w),
312         h=np.full(n, h),
313         z=np.full(n, z),
314         tower_xy=np.asarray(tower_xy, dtype=float),
315     )
316
317
318 def make_ring_variable_config(

```

```

319     tower_xy: np.ndarray,
320     ring_params: List[Tuple[float, float, float]],
321     gap: float = 5.0,
322 ) -> FieldConfig:
323     """ring_params: list of (radius, width(=height), install_z)."""
324     pts, ws, hs, zs = [], [], [], []
325     for r, wh, z in ring_params:
326         if r < CLEAR_R or r > FIELD_R:
327             continue
328         dr = wh + gap
329         n = max(6, int(math.floor(2 * math.pi * r / dr)))
330         for k in range(n):
331             th = 2 * math.pi * k / n
332             x = tower_xy[0] + r * math.cos(th)
333             y = tower_xy[1] + r * math.sin(th)
334             if x**2 + y**2 <= FIELD_R**2 and (x - tower_xy[0])**2 + (y - tower_xy[1])**2 >= CLEAR_R**2:
335                 pts.append([x, y])
336                 ws.append(wh)
337                 hs.append(wh)
338                 zs.append(z)
339     xy = np.asarray(pts, dtype=float) if pts else np.zeros((0, 2))
340     return FieldConfig(
341         xy=xy,
342         w=np.asarray(ws, dtype=float),
343         h=np.asarray(hs, dtype=float),
344         z=np.asarray(zs, dtype=float),
345         tower_xy=np.asarray(tower_xy, dtype=float),
346     )

```

Listing 1: heliostat_field.py 完整代码

C 问题二、三优化与导出程序（finalize2.py）

```
1  # -*- coding: utf-8 -*-
2  """Full-eval only finalize for Q2/Q3 meeting 60 MW."""
3  import json
4  import sys
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10
11  ROOT = Path(__file__).resolve().parents[1]
12  sys.path.insert(0, str(ROOT / "code"))
13  from heliostat_field import * # noqa
14  from solve_all import ( # noqa
15      annual_table,
16      compare_bar,
17      monthly_table,
18      plot_eta_map,
19      plot_layout,
20      plot_monthly,
21      savefig,
22      TAB,
23  )
24
25  plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
26  plt.rcParams["axes.unicode_minus"] = False
27
28
29  def main():
30      xy1 = load_attachment(str(ROOT / "data" / "attachment.xlsx"))
31      cfg1 = make_uniform_config(xy1, 6, 6, 4, np.array([0.0, 0.0]))
32      r1 = evaluate_annual(cfg1, fast=False)
33      monthly_table(r1).to_excel(TAB / "表1_问题1月平均.xlsx", index=False)
34      annual_table(r1).to_excel(TAB / "表2_问题1年平均.xlsx", index=False)
35
36      best2 = None
37      # full grid (compact)
38      for ty in [-80, -60, -40, -20, 0, 20]:
39          for tx in [0.0]:
40              for w in [6.5, 7.0, 7.5, 8.0]:
41                  z = float(min(6.0, max(w / 2 + 0.4, 4.0)))
42                  tower = np.array([tx, float(ty)])
43                  if tx * tx + ty * ty > (FIELD_R - 5) ** 2:
44                      continue
45                  xy = concentric_layout(tower, w, gap=5.0)
46                  cfg = make_uniform_config(xy, w, w, z, tower)
47                  res = evaluate_annual(cfg, fast=False)
48                  ok = res["power_mw"] >= 60
49                  # maximize unit among feasible; else maximize power
50                  key = (1, res["unit_kw"]) if ok else (0, res["power_mw"])
51                  print(f"Q2 w={w} ty={ty} n={res['n']} P={res['power_mw']:.2f} u={res['unit_kw']:.4f} ok={ok}")
52                  if best2 is None or key > best2[0]:
53                      best2 = (key, cfg, res, dict(tx=tx, ty=ty, w=w, z=z))
54
55      cfg2, r2, meta2 = best2[1], best2[2], best2[3]
56      print("Q2 BEST", meta2, r2["power_mw"], r2["unit_kw"])
57
58      # Q3 around same tower: try to beat q2 unit with P>=60
59      tower = cfg2.tower_xy.copy()
60      best3 = None
61      for scale in [0.95, 1.0, 1.05, 1.1, 1.15]:
62          for z0 in [4.0, 4.5, 5.0]:
```

```

63     rings = []
64     r = CLEAR_R + 2
65     k = 0
66     while r < FIELD_R - 2:
67         wh = float(np.clip(meta2["w"] * scale * (1.06 - 0.012 * k), 3.0, 8.0))
68         z = float(np.clip(max(z0, wh / 2 + 0.35), 2.0, 6.0))
69         rings.append((r, wh, z))
70         r += wh + 5.0
71         k += 1
72     cfg = make_ring_variable_config(tower, rings, gap=5.0)
73     if len(cfg.xy) < 1000:
74         continue
75     res = evaluate_annual(cfg, fast=False)
76     ok = res["power_mw"] >= 60
77     key = (1, res["unit_kw"]) if ok else (0, res["power_mw"])
78     print(f"Q3 scale={scale:.2f} z0={z0} n={res['n']} P={res['power_mw']:.2f} u={res['unit_kw']:.4f}")
79     if best3 is None or key > best3[0]:
80         best3 = (key, cfg, res)
81
82     cfg3, r3 = best3[1], best3[2]
83     print("Q3 BEST", r3["n"], r3["power_mw"], r3["unit_kw"])
84
85     # exports
86     monthly_table(r2).to_excel(TAB / "表1_问题2月平均.xlsx", index=False)
87     annual_table(r2).to_excel(TAB / "表2_问题2年平均.xlsx", index=False)
88     pd.DataFrame([
89         "吸收塔x坐标(m)": round(float(cfg2.tower_xy[0]), 3),
90         "吸收塔y坐标(m)": round(float(cfg2.tower_xy[1]), 3),
91         "定日镜尺寸(m)": round(float(cfg2.w[0]), 3),
92         "安装高度(m)": round(float(cfg2.z[0]), 3),
93         "定日镜数目": int(r2["n"]),
94         "定日镜总面积(m2)": round(r2["area"], 2),
95     ]).to_excel(TAB / "表3_问题2设计参数.xlsx", index=False)
96     with pd.ExcelWriter(ROOT / "result2.xlsx", engine="openpyxl") as w:
97         pd.DataFrame({
98             "吸收塔x坐标(m)": [cfg2.tower_xy[0]],
99             "吸收塔y坐标(m)": [cfg2.tower_xy[1]],
100            "定日镜宽(m)": [cfg2.w[0]],
101            "定日镜高(m)": [cfg2.h[0]],
102            "安装高度(m)": [cfg2.z[0]],
103            "数目": [len(cfg2.xy)],
104        }).to_excel(w, sheet_name="参数", index=False)
105     pd.DataFrame({"x坐标 (m)": cfg2.xy[:, 0], "y坐标 (m)": cfg2.xy[:, 1]}).to_excel(
106         w, sheet_name="定日镜位置", index=False)
107     )
108
109     monthly_table(r3).to_excel(TAB / "表1_问题3月平均.xlsx", index=False)
110     annual_table(r3).to_excel(TAB / "表2_问题3年平均.xlsx", index=False)
111     pd.DataFrame([
112         "吸收塔x坐标(m)": round(float(cfg3.tower_xy[0]), 3),
113         "吸收塔y坐标(m)": round(float(cfg3.tower_xy[1]), 3),
114         "定日镜数目": int(r3["n"]),
115         "定日镜总面积(m2)": round(r3["area"], 2),
116         "尺寸范围(m)": f"{cfg3.w.min():.2f}~{cfg3.w.max():.2f}",
117         "高度范围(m)": f"{cfg3.z.min():.2f}~{cfg3.z.max():.2f}",
118     ]).to_excel(TAB / "表3_问题3设计参数.xlsx", index=False)
119     with pd.ExcelWriter(ROOT / "result3.xlsx", engine="openpyxl") as w:
120         pd.DataFrame({
121             "吸收塔x坐标(m)": [cfg3.tower_xy[0]],
122             "吸收塔y坐标(m)": [cfg3.tower_xy[1]],
123             "数目": [len(cfg3.xy)],
124         }).to_excel(w, sheet_name="参数", index=False)
125     pd.DataFrame({
126         "x坐标 (m)": cfg3.xy[:, 0],

```

```

127     "y坐标 (m)": cfg3.xy[:, 1],
128     "定日镜尺寸 (m)": cfg3.w,
129     "安装高度 (m)": cfg3.z,
130     }).to_excel(w, sheet_name="定日镜", index=False)
131
132 plot_layout(cfg2.xy, cfg2.tower_xy, "fig06_q2_layout.png", "问题二: 优化后定日镜场布局")
133 plot_monthly(r2, "fig07_q2_monthly.png", "问题二: 各月平均效率")
134 plot_eta_map(cfg2, "fig08_q2_eta_map.png", "问题二: 春分正午光学效率分布")
135 plot_layout(cfg3.xy, cfg3.tower_xy, "fig10_q3_layout.png", "问题三: 变尺寸/高度优化布局")
136 plot_monthly(r3, "fig11_q3_monthly.png", "问题三: 各月平均效率")
137 plot_eta_map(cfg3, "fig12_q3_eta_map.png", "问题三: 春分正午光学效率分布")
138 rr = np.linalg.norm(cfg3.xy - cfg3.tower_xy, axis=1)
139 fig, ax = plt.subplots(figsize=(7.5, 4))
140 ax.scatter(rr, cfg3.w, s=8, alpha=0.7)
141 ax.set_xlabel("到吸收塔距离 / m")
142 ax.set_ylabel("定日镜边长 / m")
143 ax.set_title("问题三: 定日镜尺寸随径向距离的变化")
144 ax.grid(True, alpha=0.3)
145 savefig("fig13_q3_size_radius.png")
146 compare_bar(r1, r2, r3)
147
148 summary = {
149     "q1": {k: r1[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
150     "q2": {
151         **{k: r2[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
152         "tower": [float(cfg2.tower_xy[0]), float(cfg2.tower_xy[1])],
153         "w": float(cfg2.w[0]),
154         "z": float(cfg2.z[0]),
155     },
156     "q3": {
157         **{k: r3[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
158         "tower": [float(cfg3.tower_xy[0]), float(cfg3.tower_xy[1])],
159         "w_min": float(cfg3.w.min()),
160         "w_max": float(cfg3.w.max()),
161         "z_min": float(cfg3.z.min()),
162         "z_max": float(cfg3.z.max()),
163     },
164     "monthly_q1": r1["monthly"],
165     "monthly_q2": r2["monthly"],
166     "monthly_q3": r3["monthly"],
167 }
168 with open(ROOT / "code" / "results.json", "w", encoding="utf-8") as f:
169     json.dump(summary, f, ensure_ascii=False, indent=2)
170 print(json.dumps({k: summary[k] for k in ["q1", "q2", "q3"]}, ensure_ascii=False, indent=2))
171
172
173 if __name__ == "__main__":
174     main()

```

Listing 2: finalize2.py 完整代码

D 作图与流程辅助程序 (solve_all.py)

```
1  # -*- coding: utf-8 -*-
2  """Solve 2023 CUMCM A problems 1-3, export tables/figures/xlsx."""
3  from __future__ import annotations
4
5  import json
6  import os
7  import sys
8  from pathlib import Path
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13
14 ROOT = Path(__file__).resolve().parents[1]
15 sys.path.insert(0, str(ROOT / "code"))
16
17 from heliostat_field import ( # noqa: E402
18     CLEAR_R,
19     FIELD_R,
20     concentric_layout,
21     evaluate_annual,
22     load_attachment,
23     make_ring_variable_config,
24     make_uniform_config,
25     sun_position,
26 )
27
28 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
29 plt.rcParams["axes.unicode_minus"] = False
30
31 FIG = ROOT / "figures"
32 PIC = ROOT / "图片"
33 TAB = ROOT / "表格"
34 for d in (FIG, PIC, TAB, ROOT / "data"):
35     d.mkdir(exist_ok=True)
36
37
38 def savefig(name: str):
39     p1 = FIG / name
40     p2 = PIC / name
41     plt.tight_layout()
42     plt.savefig(p1, dpi=160, bbox_inches="tight")
43     plt.savefig(p2, dpi=160, bbox_inches="tight")
44     plt.close()
45     print("fig:", p1)
46
47
48 def monthly_table(res: dict) -> pd.DataFrame:
49     rows = []
50     for r in res["monthly"]:
51         rows.append(
52             {
53                 "日期": f"{r['month']}月21日",
54                 "平均光学效率": round(r["eta"], 4),
55                 "平均余弦效率": round(r["cos"], 4),
56                 "平均阴影遮挡效率": round(r["sb"], 4),
57                 "平均截断效率": round(r["trunc"], 4),
58                 "单位面积镜面平均输出热功率(kW/m2)": round(r["unit_kw"], 4),
59             }
60         )
61     return pd.DataFrame(rows)
62
```

```

63
64 def annual_table(res: dict) -> pd.DataFrame:
65     return pd.DataFrame(
66         [
67             {
68                 "年平均光学效率": round(res["eta"], 4),
69                 "年平均余弦效率": round(res["cos"], 4),
70                 "年平均阴影遮挡效率": round(res["sb"], 4),
71                 "年平均截断效率": round(res["trunc"], 4),
72                 "年平均输出热功率(MW)": round(res["power_mw"], 4),
73                 "单位面积镜面年平均输出热功率(kW/m2)": round(res["unit_kw"], 4),
74             }
75         ]
76     )
77
78
79 def plot_sun():
80     months = range(1, 13)
81     hours = [9, 10.5, 12, 13.5, 15]
82     fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
83     for h in hours:
84         al, ga = [], []
85         for m in months:
86             a, g, _ = sun_position(m, h)
87             al.append(np.degrees(a))
88             ga.append(np.degrees(g))
89         axes[0].plot(list(months), al, marker="o", label=f"{h:g}时")
90         axes[1].plot(list(months), ga, marker="o", label=f"{h:g}时")
91     axes[0].set_xlabel("月份")
92     axes[0].set_ylabel("太阳高度角 (°)")
93     axes[0].set_title("太阳高度角年变化")
94     axes[0].legend(fontsize=8)
95     axes[0].grid(True, alpha=0.3)
96     axes[1].set_xlabel("月份")
97     axes[1].set_ylabel("太阳方位角 (°)")
98     axes[1].set_title("太阳方位角年变化")
99     axes[1].legend(fontsize=8)
100    axes[1].grid(True, alpha=0.3)
101    savefig("fig01_sun_angles.png")
102
103
104 def plot_layout(xy, tower, name, title):
105     fig, ax = plt.subplots(figsize=(6.2, 6.2))
106     th = np.linspace(0, 2 * np.pi, 400)
107     ax.plot(FIELD_R * np.cos(th), FIELD_R * np.sin(th), "k-", lw=1.2)
108     ax.plot(CLEAR_R * np.cos(th) + tower[0], CLEAR_R * np.sin(th) + tower[1], "k--", lw=0.8, alpha=0.6)
109     ax.scatter(xy[:, 0], xy[:, 1], s=4, c="#1f77b4", alpha=0.75)
110     ax.scatter([tower[0]], [tower[1]], s=80, c="crimson", marker="^", label="吸收塔", zorder=5)
111     ax.set_aspect("equal")
112     ax.set_xlim(-370, 370)
113     ax.set_ylim(-370, 370)
114     ax.set_xlabel("x / m (东)")
115     ax.set_ylabel("y / m (北)")
116     ax.set_title(title)
117     ax.legend(loc="upper right")
118     ax.grid(True, alpha=0.25)
119     savefig(name)
120
121
122 def plot_monthly(res, name, title):
123     m = [r["month"] for r in res["monthly"]]
124     fig, ax = plt.subplots(figsize=(8.5, 4.5))
125     ax.plot(m, [r["eta"] for r in res["monthly"]], "o-", label="光学效率")
126     ax.plot(m, [r["cos"] for r in res["monthly"]], "s-", label="余弦效率")

```

```

127     ax.plot(m, [r["sb"] for r in res["monthly"]], "--", label="阴影遮挡效率")
128     ax.plot(m, [r["trunc"] for r in res["monthly"]], "d-", label="截断效率")
129     ax.set_xlabel("月份")
130     ax.set_ylabel("效率")
131     ax.set_ylim(0.4, 1.05)
132     ax.set_title(title)
133     ax.legend()
134     ax.grid(True, alpha=0.3)
135     savefig(name)
136
137
138 def plot_eta_map(cfg, name, title):
139     # use spring equinox noon as representative optical efficiency map
140     from heliostat_field import evaluate_instant
141
142     a, g, dni = sun_position(3, 12)
143     r = evaluate_instant(cfg, a, g, dni)
144     eta = r["eta_i"]
145     fig, ax = plt.subplots(figsize=(6.4, 5.6))
146     sc = ax.scatter(cfg.xy[:, 0], cfg.xy[:, 1], c=eta, s=8, cmap="viridis")
147     ax.scatter([cfg.tower_xy[0]], [cfg.tower_xy[1]], c="r", s=70, marker="^")
148     th = np.linspace(0, 2 * np.pi, 300)
149     ax.plot(FIELD_R * np.cos(th), FIELD_R * np.sin(th), "k-", lw=1)
150     plt.colorbar(sc, ax=ax, label="光学效率")
151     ax.set_aspect("equal")
152     ax.set_title(title)
153     ax.set_xlabel("x / m")
154     ax.set_ylabel("y / m")
155     savefig(name)
156
157
158 def solve_q1():
159     print("=== Q1 ===")
160     xy = load_attachment(str(ROOT / "data" / "attachment.xlsx"))
161     cfg = make_uniform_config(xy, 6.0, 6.0, 4.0, np.array([0.0, 0.0]))
162     res = evaluate_annual(cfg)
163     print(annual_table(res))
164     monthly_table(res).to_excel(TAB / "表1_问题1月平均.xlsx", index=False)
165     annual_table(res).to_excel(TAB / "表2_问题1年平均.xlsx", index=False)
166     plot_layout(xy, cfg.tower_xy, "fig02_q1_layout.png", "问题一：定日镜场初始布局（塔在圆心）")
167     plot_monthly(res, "fig03_q1_monthly.png", "问题一：各月平均效率")
168     plot_eta_map(cfg, "fig04_q1_eta_map.png", "问题一：春分正午光学效率分布")
169     # per-mirror annual rough: average of 12 noon snapshots
170     from heliostat_field import evaluate_instant
171
172     acc = np.zeros(len(xy))
173     for m in range(1, 13):
174         a, g, dni = sun_position(m, 12)
175         acc += evaluate_instant(cfg, a, g, dni)["eta_i"]
176     acc /= 12
177     fig, ax = plt.subplots(figsize=(9, 3.8))
178     ax.plot(acc, lw=0.8)
179     ax.set_xlabel("定日镜序号（由内层起）")
180     ax.set_ylabel("近似年平均光学效率")
181     ax.set_title("问题一：各定日镜年平均光学效率")
182     ax.grid(True, alpha=0.3)
183     savefig("fig05_q1_eta_series.png")
184     return cfg, res
185
186
187 def _q2_score(power: float, unit: float) -> float:
188     """Prefer feasible (>=60 MW), then maximize unit power."""
189     if power >= 60:
190         return 1000.0 + unit

```

```

191     return power + 0.01 * unit # climb toward rated power first
192
193
194 def search_q2():
195     print("=== Q2 search ===")
196     best = None
197     candidates = []
198     for ty in np.linspace(-140, 0, 8):
199         for tx in np.linspace(-20, 20, 3):
200             for w in [4.0, 5.0, 6.0, 7.0, 8.0]:
201                 for z in [4.0, 5.0, 6.0]:
202                     if z <= w / 2 + 0.05:
203                         continue
204                     candidates.append((tx, ty, w, z))
205     print("candidates", len(candidates))
206     results = []
207     for i, (tx, ty, w, z) in enumerate(candidates):
208         if tx**2 + ty**2 > (FIELD_R - 5) ** 2:
209             continue
210         tower = np.array([tx, ty])
211         xy = concentric_layout(tower, w, r_min=CLEAR_R, r_max=FIELD_R, gap=5.0)
212         if len(xy) < 200:
213             continue
214         cfg = make_uniform_config(xy, w, w, z, tower)
215         res = evaluate_annual(cfg, fast=True)
216         score = _q2_score(res["power_mw"], res["unit_kw"])
217         item = dict(tx=tx, ty=ty, w=w, z=z, n=res["n"], power=res["power_mw"], unit=res["unit_kw"], score=score)
218         results.append(item)
219         if best is None or item["score"] > best["score"]:
220             best = item
221             print(f" best@{i}: n={item['n']} P={item['power']:.2f} unit={item['unit']:.4f} w={w} z={z} tower=({tx:.1f},{ty:.1f})")
222
223 # among top power candidates, refine for unit power under power constraint
224 results.sort(key=lambda d: d["score"], reverse=True)
225 pool = results[:20]
226 best_full = None
227 for item in pool:
228     for dtx in (-10, 0, 10):
229         for dty in (-15, 0, 15):
230             for dw in (-0.5, 0, 0.5, 1.0):
231                 tx, ty = item["tx"] + dtx, item["ty"] + dty
232                 w = float(np.clip(item["w"] + dw, 2.0, 8.0))
233                 z = float(np.clip(max(item["z"], w / 2 + 0.3), 2.0, 6.0))
234                 if tx**2 + ty**2 > (FIELD_R - 5) ** 2:
235                     continue
236                 tower = np.array([tx, ty])
237                 for gap in [5.0, 5.5]:
238                     xy = concentric_layout(tower, w, gap=gap)
239                     if len(xy) < 200:
240                         continue
241                     res = evaluate_annual(make_uniform_config(xy, w, w, z, tower), fast=True)
242                     score = _q2_score(res["power_mw"], res["unit_kw"])
243                     cand = dict(tx=tx, ty=ty, w=w, z=z, gap=gap, n=res["n"], power=res["power_mw"], unit=res["unit_kw"], score=
score)
244                     if best_full is None or cand["score"] > best_full["score"]:
245                         best_full = cand
246     print(" refine-fast", best_full)
247 # full evaluation around best_full
248 seeds = [best_full, best]
249 best = None
250 for seed in seeds:
251     if seed is None:
252         continue
253     for w in np.linspace(max(3.0, seed["w"] - 1), min(8.0, seed["w"] + 1), 5):

```

```

254     for ty in np.linspace(seed["ty"] - 20, seed["ty"] + 20, 5):
255         tx = seed["tx"]
256         z = float(np.clip(max(seed.get("z", 4), w / 2 + 0.3), 2.0, 6.0))
257         if tx**2 + ty**2 > (FIELD_R - 5) ** 2:
258             continue
259         tower = np.array([tx, ty])
260         xy = concentric_layout(tower, w, gap=5.0)
261         cfg = make_uniform_config(xy, w, w, z, tower)
262         res = evaluate_annual(cfg, fast=False)
263         score = _q2_score(res["power_mw"], res["unit_kw"])
264         cand = dict(tx=tx, ty=ty, w=float(w), z=z, gap=5.0, n=res["n"], power=res["power_mw"], unit=res["unit_kw"], score=
score)
265         if best is None or cand["score"] > best["score"]:
266             best = cand
267             print(f"    full: n={cand['n']} P={cand['power']:.2f} unit={cand['unit']:.4f} w={cand['w']:.2f}")
268
269 # last resort: maximize area with w=8
270 if best is None or best["power"] < 60:
271     for ty in np.linspace(-80, 40, 7):
272         for w in [7.0, 7.5, 8.0]:
273             z = min(6.0, w / 2 + 0.5)
274             tower = np.array([0.0, ty])
275             if tower[0] ** 2 + tower[1] ** 2 > (FIELD_R - 5) ** 2:
276                 continue
277             xy = concentric_layout(tower, w, gap=5.0)
278             cfg = make_uniform_config(xy, w, w, z, tower)
279             res = evaluate_annual(cfg, fast=False)
280             score = _q2_score(res["power_mw"], res["unit_kw"])
281             cand = dict(tx=0.0, ty=float(ty), w=float(w), z=float(z), gap=5.0, n=res["n"], power=res["power_mw"], unit=res["
unit_kw"], score=score)
282             if best is None or cand["score"] > best["score"]:
283                 best = cand
284                 print(f"    rescue: n={cand['n']} P={cand['power']:.2f} unit={cand['unit']:.4f}")
285
286 print("Q2 best", best)
287 tower = np.array([best["tx"], best["ty"]])
288 xy = concentric_layout(tower, best["w"], gap=best.get("gap", 5.0))
289 cfg = make_uniform_config(xy, best["w"], best["w"], best["z"], tower)
290 res = evaluate_annual(cfg, fast=False)
291 return cfg, res, best, results
292
293
294 def solve_q2():
295     cfg, res, best, hist = search_q2()
296     monthly_table(res).to_excel(TAB / "表1_问题2月平均.xlsx", index=False)
297     annual_table(res).to_excel(TAB / "表2_问题2年平均.xlsx", index=False)
298     pd.DataFrame(
299         [
300             {
301                 "吸收塔x坐标(m)": round(best["tx"], 3),
302                 "吸收塔y坐标(m)": round(best["ty"], 3),
303                 "定日镜尺寸(m)": round(best["w"], 3),
304                 "安装高度(m)": round(best["z"], 3),
305                 "定日镜数目": int(res["n"]),
306                 "定日镜总面积(m2)": round(res["area"], 2),
307             }
308         ]
309     ).to_excel(TAB / "表3_问题2设计参数.xlsx", index=False)
310
311 # result2.xlsx
312 df_pos = pd.DataFrame(
313     {
314         "吸收塔x坐标 (m)": [cfg.tower_xy[0]] + [np.nan] * (len(cfg.xy) - 1),
315         "吸收塔y坐标 (m)": [cfg.tower_xy[1]] + [np.nan] * (len(cfg.xy) - 1),

```

```

316         "定日镜尺寸 (m)": [cfg.w[0]] + [np.nan] * (len(cfg.xy) - 1) if False else cfg.w,
317     }
318 )
319 out = pd.DataFrame(
320     {
321         "吸收塔x坐标 (m)": [float(cfg.tower_xy[0])] + [None] * (len(cfg.xy) - 1),
322         "吸收塔y坐标 (m)": [float(cfg.tower_xy[1])] + [None] * (len(cfg.xy) - 1),
323         "定日镜尺寸 (m)": [float(cfg.w[0])] + [None] * (len(cfg.xy) - 1),
324         "定日镜高度 (m)": [float(cfg.h[0])] + [None] * (len(cfg.xy) - 1),
325         "安装高度 (m)": [float(cfg.z[0])] + [None] * (len(cfg.xy) - 1),
326         "定日镜x坐标 (m)": cfg.xy[:, 0],
327         "定日镜y坐标 (m)": cfg.xy[:, 1],
328     }
329 )
330 # simpler standard template
331 out2 = pd.DataFrame(
332     {
333         "x坐标 (m)": cfg.xy[:, 0],
334         "y坐标 (m)": cfg.xy[:, 1],
335     }
336 )
337 with pd.ExcelWriter(ROOT / "result2.xlsx", engine="openpyxl") as writer:
338     pd.DataFrame(
339         {
340             "吸收塔x坐标(m)": [cfg.tower_xy[0]],
341             "吸收塔y坐标(m)": [cfg.tower_xy[1]],
342             "定日镜宽(m)": [cfg.w[0]],
343             "定日镜高(m)": [cfg.h[0]],
344             "安装高度(m)": [cfg.z[0]],
345             "数目": [len(cfg.xy)],
346         }
347     ).to_excel(writer, sheet_name="参数", index=False)
348     out2.to_excel(writer, sheet_name="定日镜位置", index=False)
349
350 plot_layout(cfg.xy, cfg.tower_xy, "fig06_q2_layout.png", "问题二: 优化后定日镜场布局")
351 plot_monthly(res, "fig07_q2_monthly.png", "问题二: 各月平均效率")
352 plot_eta_map(cfg, "fig08_q2_eta_map.png", "问题二: 春分正午光学效率分布")
353
354 # random feasible comparison
355 rng = np.random.default_rng(42)
356 units = []
357 for _ in range(40):
358     tx = float(rng.uniform(-80, 80))
359     ty = float(rng.uniform(-200, -20))
360     w = float(rng.uniform(3, 6))
361     z = float(rng.uniform(max(w / 2 + 0.3, 2.5), 6))
362     if tx**2 + ty**2 > (FIELD_R - 5) ** 2:
363         continue
364     xy = concentric_layout(np.array([tx, ty]), w, gap=5.0)
365     if len(xy) < 300:
366         continue
367     r = evaluate_annual(make_uniform_config(xy, w, w, z, np.array([tx, ty])), fast=True)
368     if r["power_mw"] >= 50:
369         units.append(r["unit_kw"])
370 fig, ax = plt.subplots(figsize=(8, 3.8))
371 if units:
372     ax.stem(range(len(units)), units, linefmt="C0-", markerfmt="C0o", basefmt=" ")
373 ax.axhline(res["unit_kw"], color="r", ls="--", label=f"最优 {res['unit_kw']:.4f}")
374 ax.set_xlabel("随机可行解序号")
375 ax.set_ylabel("单位面积年平均输出热功率")
376 ax.set_title("问题二: 随机可行解与最优解比较")
377 ax.legend()
378 savefig("fig09_q2_random.png")
379 return cfg, res, best

```

```

380
381
382 def solve_q3(q2_cfg, q2_best):
383     print("=== Q3 ===")
384     tower = q2_cfg.tower_xy.copy()
385     # rings with variable size: inner smaller? actually outer often larger gap; use decreasing size outward for intercept
386     best = None
387     best_cfg = None
388     best_res = None
389     for scale in [0.85, 0.9, 0.95, 1.0, 1.05]:
390         for z0 in [3.2, 3.8, 4.2, 4.8]:
391             rings = []
392             r = CLEAR_R + 2.0
393             k = 0
394             while r < FIELD_R - 2:
395                 # size grows slightly then shrinks
396                 wh = float(np.clip(q2_best["w"] * scale * (1.05 - 0.012 * k), 2.0, 8.0))
397                 z = float(np.clip(max(z0, wh / 2 + 0.25) + 0.03 * k, 2.0, 6.0))
398                 rings.append((r, wh, z))
399                 r += wh + 5.0
400                 k += 1
401             cfg = make_ring_variable_config(tower, rings, gap=5.0)
402             if len(cfg.xy) < 300:
403                 continue
404             res = evaluate_annual(cfg, fast=True)
405             ok = res["power_mw"] >= 58
406             score = res["unit_kw"] if ok else res["unit_kw"] - 10 * (60 - res["power_mw"])
407             if best is None or score > best:
408                 best = score
409                 best_cfg, best_res = cfg, res
410                 print(f" Q3-fast: n={res['n']} P={res['power_mw']:.2f} unit={res['unit_kw']:.4f}")
411
412     best_res = evaluate_annual(best_cfg, fast=False)
413     # if still < 60, densify
414     if best_res["power_mw"] < 60:
415         rings = []
416         r = CLEAR_R + 1.5
417         k = 0
418         while r < FIELD_R - 1:
419             wh = float(np.clip(q2_best["w"] * 1.1 * (1.02 - 0.008 * k), 2.5, 8.0))
420             z = float(np.clip(max(4.0, wh / 2 + 0.3), 2.0, 6.0))
421             rings.append((r, wh, z))
422             r += wh + 4.8
423             k += 1
424         best_cfg = make_ring_variable_config(tower, rings, gap=4.8)
425         best_res = evaluate_annual(best_cfg, fast=False)
426
427     monthly_table(best_res).to_excel(TAB / "表1_问题3月平均.xlsx", index=False)
428     annual_table(best_res).to_excel(TAB / "表2_问题3年平均.xlsx", index=False)
429     pd.DataFrame(
430         [
431             {
432                 "吸收塔x坐标(m)": round(float(tower[0]), 3),
433                 "吸收塔y坐标(m)": round(float(tower[1]), 3),
434                 "定日镜数目": int(best_res["n"]),
435                 "定日镜总面积(m2)": round(best_res["area"], 2),
436                 "尺寸范围(m)": f"{best_cfg.w.min():.2f}~{best_cfg.w.max():.2f}",
437                 "高度范围(m)": f"{best_cfg.z.min():.2f}~{best_cfg.z.max():.2f}",
438             }
439         ]
440     ).to_excel(TAB / "表3_问题3设计参数.xlsx", index=False)
441
442     with pd.ExcelWriter(ROOT / "result3.xlsx", engine="openpyxl") as writer:
443         pd.DataFrame(

```

```

444         {
445             "吸收塔x坐标(m)": [tower[0]],
446             "吸收塔y坐标(m)": [tower[1]],
447             "数目": [len(best_cfg.xy)],
448         }
449     ).to_excel(writer, sheet_name="参数", index=False)
450     pd.DataFrame(
451         {
452             "x坐标 (m)": best_cfg.xy[:, 0],
453             "y坐标 (m)": best_cfg.xy[:, 1],
454             "定日镜尺寸 (m)": best_cfg.w,
455             "安装高度 (m)": best_cfg.z,
456         }
457     ).to_excel(writer, sheet_name="定日镜", index=False)
458
459     plot_layout(best_cfg.xy, tower, "fig10_q3_layout.png", "问题三: 变尺寸/高度优化布局")
460     plot_monthly(best_res, "fig11_q3_monthly.png", "问题三: 各月平均效率")
461     plot_eta_map(best_cfg, "fig12_q3_eta_map.png", "问题三: 春分正午光学效率分布")
462
463     # size vs radius
464     rr = np.linalg.norm(best_cfg.xy - tower, axis=1)
465     fig, ax = plt.subplots(figsize=(7.5, 4))
466     ax.scatter(rr, best_cfg.w, s=8, alpha=0.7)
467     ax.set_xlabel("到吸收塔距离 / m")
468     ax.set_ylabel("定日镜边长 / m")
469     ax.set_title("问题三: 定日镜尺寸随径向距离的变化")
470     ax.grid(True, alpha=0.3)
471     savefig("fig13_q3_size_radius.png")
472     return best_cfg, best_res
473
474
475 def compare_bar(r1, r2, r3):
476     labels = ["光学效率", "余弦效率", "遮挡效率", "截断效率", "单位功率"]
477     v1 = [r1["eta"], r1["cos"], r1["sb"], r1["trunc"], r1["unit_kw"] / max(r1["unit_kw"], 1e-9)]
478     # normalize unit to relative for plot; also separate power plot
479     fig, axes = plt.subplots(1, 2, figsize=(10, 4.2))
480     x = np.arange(4)
481     w = 0.25
482     axes[0].bar(x - w, [r1["eta"], r1["cos"], r1["sb"], r1["trunc"]], w, label="问题一")
483     axes[0].bar(x, [r2["eta"], r2["cos"], r2["sb"], r2["trunc"]], w, label="问题二")
484     axes[0].bar(x + w, [r3["eta"], r3["cos"], r3["sb"], r3["trunc"]], w, label="问题三")
485     axes[0].set_xticks(x)
486     axes[0].set_xticklabels(["光学", "余弦", "遮挡", "截断"])
487     axes[0].set_ylim(0, 1.05)
488     axes[0].legend()
489     axes[0].set_title("年均效率对比")
490     axes[0].grid(True, axis="y", alpha=0.3)
491
492     axes[1].bar(
493         ["问题一", "问题二", "问题三"],
494         [r1["unit_kw"], r2["unit_kw"], r3["unit_kw"]],
495         color=["#4c72b0", "#55a868", "#c44e52"],
496     )
497     axes[1].set_ylabel("kW/m²")
498     axes[1].set_title("单位镜面面积年平均输出热功率")
499     axes[1].grid(True, axis="y", alpha=0.3)
500     savefig("fig14_compare.png")
501
502     fig, ax = plt.subplots(figsize=(6, 4))
503     ax.bar(
504         ["问题一", "问题二", "问题三"],
505         [r1["power_mw"], r2["power_mw"], r3["power_mw"]],
506         color=["#4c72b0", "#55a868", "#c44e52"],
507     )

```

```

508     ax.axhline(60, color="k", ls="--", label="额定60 MW")
509     ax.set_ylabel("MW")
510     ax.set_title("年平均输出热功率对比")
511     ax.legend()
512     ax.grid(True, axis="y", alpha=0.3)
513     savefig("fig15_power_compare.png")
514
515
516 def main():
517     plot_sun()
518     cfg1, r1 = solve_q1()
519     cfg2, r2, best2 = solve_q2()
520     cfg3, r3 = solve_q3(cfg2, best2)
521     compare_bar(r1, r2, r3)
522
523     summary = {
524         "q1": {k: r1[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
525         "q2": {
526             **{k: r2[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
527             "tower": [float(cfg2.tower_xy[0]), float(cfg2.tower_xy[1])],
528             "w": float(cfg2.w[0]),
529             "z": float(cfg2.z[0]),
530         },
531         "q3": {
532             **{k: r3[k] for k in ["eta", "cos", "sb", "trunc", "power_mw", "unit_kw", "n", "area"]},
533             "tower": [float(cfg3.tower_xy[0]), float(cfg3.tower_xy[1])],
534             "w_min": float(cfg3.w.min()),
535             "w_max": float(cfg3.w.max()),
536             "z_min": float(cfg3.z.min()),
537             "z_max": float(cfg3.z.max()),
538         },
539     }
540     with open(ROOT / "code" / "results.json", "w", encoding="utf-8") as f:
541         json.dump(summary, f, ensure_ascii=False, indent=2)
542     print("=== SUMMARY ===")
543     print(json.dumps(summary, ensure_ascii=False, indent=2))
544
545
546 if __name__ == "__main__":
547     main()

```

Listing 3: solve_all.py 完整代码